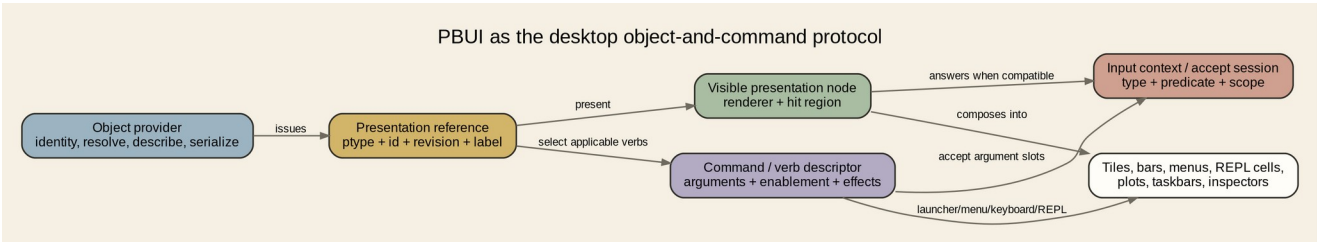


go-go-wm

Engineering Guide

A presentation-oriented, scriptable desktop runtime on X11

Code review • resize-performance redesign • PBUI widget architecture • REPL as operating-environment control plane



REVIEW BASELINE	main at 5b73c9f37c97538f6767ecdc3ece4fb599932377
DESIGN INTERVAL	GGWM-002 through GGWM-011, 18-20 July 2026
PREPARED	21 July 2026

Static code and architecture review. Performance recommendations require verification on the target display, clients, and hardware.

Contents

Right-click and update this field in Word if page numbers are not visible.

Static guide map (Word/LibreOffice may populate the field above with page numbers):

Part I. Window-manager foundations

1. What an X11 window manager actually owns
2. A systematic method for building the manager

Part II. Review of the current implementation

3. What the three-day sequence established
4. Package architecture and ownership review
5. Current strengths to preserve

Part III. Resize performance and rendering architecture

6. Trace the current divider resize before changing it
7. The target geometry transaction
8. Separate chrome from pane-sized surfaces
9. A concrete implementation plan for fast resize
10. Performance test laboratory

Part IV. PBUI and a fully scriptable surface runtime

11. Define PBUI as a protocol, not a component library
12. Commands, verbs, and one desktop command graph
13. From row/segment IR to a retained widget tree
14. One surface runtime for tiles, bars, menus, modals, and taskbars

Part V. JavaScript runtimes and the REPL as OS building blocks

15. Treat JavaScript runtimes as desktop processes
16. State, persistence, hot reload, and packages
17. The REPL as the operating-environment control plane
18. The operating-environment service map

Part VI. Engineering roadmap, tests, and intern curriculum

19. Observability and correctness architecture
 20. Phased roadmap
 21. Intern curriculum and evidence-driven labs
 22. Architectural decisions to record now
- Appendix A. Proposed protocol and API sketches
- Appendix B. Detailed resize benchmark specification
- Appendix C. ICCCM/EWMH and desktop-integration checklist
- Appendix D. Source map
- Appendix E. File-by-file review prompts for future contributors
- Closing assessment

go-go-wm should be designed as two systems that meet at a narrow boundary. The first is a predictable window manager that owns X11 geometry, focus, mapping, stacking, and protocol compliance. The second is a live, presentation-oriented desktop environment in which JavaScript runtimes create commands, objects, inspectors, and widget surfaces. The window manager must remain responsive even when the environment is complex, and the environment must remain inspectable even when user code fails.

Executive diagnosis

The project has already crossed the point where it should be treated as a small i3 clone. Its pure layout core, typed presentation protocol, broker, Goja modules, script-defined surfaces, launcher, and rich REPL form the beginning of a user-space operating environment. The unusual part is not that JavaScript can change a split ratio. The unusual part is that a value produced in one tool can remain a typed, actionable object in another tool, and that the same command can be invoked through a menu, a launcher, a key binding, an accept prompt, or the REPL.

The current implementation contains several sound architectural decisions. `wmcore` is display-independent and mutation is expressed as serializable operations. The X11 shell and each JavaScript runtime have explicit owner loops. Render hosts consume normalized snapshots rather than calling JavaScript during paint. The PBUI broker does not depend on X11. The recent focus/fullscreen refactor centralizes invariants that had previously been distributed across fields and call sites. These are the right foundations.

The primary performance problem is also clear. Divider motion changes geometry, but geometry changes still trigger size-dependent, full-frame software rendering and upload work. The 16 ms throttle lowers the number of samples, yet each accepted sample can perform whole-workspace layout, repeated tree searches, X reconfiguration, full-size RGBA painting, surface recreation, pixel conversion, and client resize work. MIT-SHM removes a transport copy; it does not remove those CPU costs. The next performance step is therefore architectural: separate geometry commits from decoration and application repaint, track desired versus applied X state, coalesce pointer motion to the latest pending state, and limit layout and paint to dirty subtrees and damaged regions.

The primary scripting problem is not API coverage. The existing `wm`, `pbui`, and `ui` modules already demonstrate a coherent vocabulary. The missing layer is a process model. Before JavaScript widgets become the menu bar, taskbar, modal system, inspector, and persistent shell, every runtime needs an identity, capability set, lifecycle, quotas, interrupt path, state schema, event-delivery policy, and debugging surface. A single trusted `rc.js` runtime is acceptable for configuration. It is not an acceptable isolation boundary for an operating environment.

The primary UI problem is invalidation granularity. The current row-and-segment IR is a good first interchange format because it keeps JavaScript out of paint paths and validates data at the boundary. It should evolve into a retained widget tree with stable keys, distinct layout and redraw invalidation, intrinsic measurement, hit testing, focus scopes, event propagation, damage tracking, and semantics. Menus, bars, popups, modals, taskbars, launchers, notebooks, and tiles should be roles of the same surface runtime rather than separate widget implementations.

The primary REPL problem is that it is already structurally important but is still scheduled like a convenience feature. A long-running cell can monopolize the runtime owner. The REPL needs interruptible cell execution, explicit cell states, runtime restart and replay, live object inspection, completion from module metadata, source-linked errors, event timelines, operation traces, persistence, and direct PBUI accept integration. The REPL should become the control plane for the desktop environment: the place where objects are inspected, commands are discovered, widgets are developed, capabilities are reviewed, runtimes are restarted, and state is replayed.

Priority order

Priority	Work	Why it comes first	Observable completion criterion
P0	Add resize tracing and current/desired geometry counters.	Performance work needs event-to-visible evidence, not only CPU profiles.	A scripted drag reports p50/p95/p99 input-to-configure latency, paint time, X request count, and coalesced samples.
P0	Decouple interactive geometry from full-frame repaint.	This is the largest remaining resize cost and blocks richer surfaces.	Resizing terminal panes does not execute a full frame raster per sample; builtin tiles may keep stale pixels until release.
P0	Add a layout snapshot and NodeID indexes.	Repeated whole-tree lookup makes every reconciliation pass harder to reason about and can become quadratic.	One layout pass yields indexed desired rectangles; reconciliation performs no recursive <code>Find</code> per frame.

Priority	Work	Why it comes first	Observable completion criterion
P0	Make REPL cells interruptible and the input queue non-blocking.	A live environment cannot allow one evaluation to freeze its control plane.	Escape/Stop interrupts an infinite loop, marks the cell cancelled, and leaves or restarts the runtime deterministically.
P0	Give broker connections unique principals and separate control replies from best-effort events.	Runtime identity and reliable control messages are prerequisites for scripts as desktop processes.	Duplicate display names cannot steal verbs; a slow event subscriber cannot lose an accept result or command reply.
P1	Split client geometry from chrome surfaces.	Fixed-height title strips and narrow borders should not require a pane-sized backing surface.	A pane width change reuses title resources and moves chrome without reallocating a full frame image.
P1	Introduce retained widget trees and fine-grained invalidation.	Full UI scriptability requires efficient nested composition, focus, and partial updates.	A counter change repaints one text node or row, not the full surface; layout is skipped when dimensions are unchanged.
P1	Introduce runtime supervision, capabilities, quotas, and serializable state.	A configurable shell and an app platform need different trust levels.	An app runtime can be stopped, restarted, inspected, denied process access, and restored from versioned state.
P1	Unify shell UI through surface roles.	Bars, taskbars, menus, modals, and notifications otherwise grow separate policy and rendering paths.	Each is created through one <code>ui.surface</code> contract with role-specific placement, stacking, focus, and lifetime policies.
P2	Build a command graph, stable object handles, universal inspectors, and persistent notebooks.	These turn the scripting layer into a coherent operating environment rather than a collection of callbacks.	The same typed command is discoverable in the launcher, a context menu, completion, key bindings, and the REPL.

Scope and evidence

This guide is a static code and design review. It examines the current repository files, the merged pull request, the repository-local GGWM design documents for 18-20 July 2026, the attached PBUI prototypes, and selected primary-source code or documentation from i3, Sway, AwesomeWM, Qtile, XMonad, X11, ICCCM/EWMH, McCLIM, HyperCard, and live Smalltalk environments. The direct Parc pages were not fetchable in the review environment, so the corresponding repository ticket archive was used instead. No claim is made that a local build or an interactive benchmark was run against the author's machine. Performance conclusions are based on the current hot path and the repository's existing profile evidence; the proposed benchmark plan is intended to verify them on real hardware.

The document uses stable source keys such as `[GG-INPUT]` and `[I3-X]`. The complete source map appears in Appendix D.

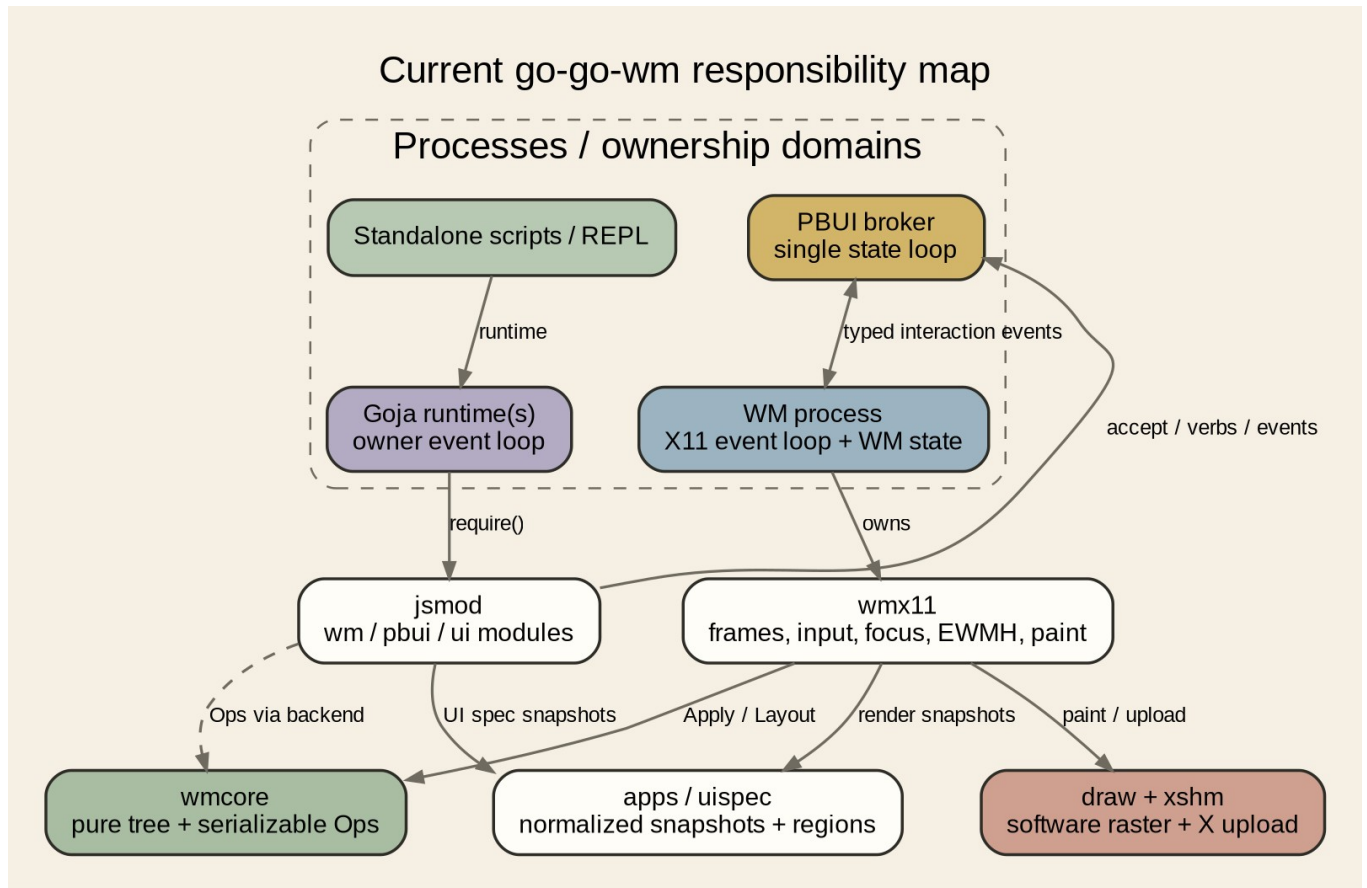


Figure 1. Current responsibility map. The pure model, the X11 shell, the broker, and JavaScript runtimes already have distinct ownership domains.

Part I. Window-manager foundations

1. What an X11 window manager actually owns

A window manager is an X11 client with one exceptional privilege: it selects `SubstructureRedirect` on a root window. Once that selection succeeds, ordinary top-level client requests to map, move, resize, raise, or lower windows are redirected to the manager as events rather than applied directly. A second manager attempting the same selection receives an error. This is the fundamental ownership boundary. The manager does not render application contents. It decides which top-level windows are visible, where their containing frames are placed, which window receives keyboard focus, and how client requests are interpreted.

The basic event path is concrete. A new application creates an unmapped top-level window and asks the X server to map it. The server sends `MapRequest` to the manager. The manager reads properties and hints, decides whether to tile, float, ignore, or specially place the client, optionally creates a frame and reparents the client into it, configures both windows, maps them, and establishes focus. From then on, the manager receives property changes, configure requests, destroy/unmap notifications, focus changes, pointer events on its own frames and dividers, and client messages implementing ICCCM or EWMH protocols.

The X server does not promise to preserve pixels through every reconfiguration. A mapped resize may lose contents and generate `Expose` events. A sophisticated client or manager repaints only the exposed region; a naive implementation repaints everything. This matters directly to go-go-wm because the manager paints its own title strips, borders, bars, menus, and builtin surfaces. Application clients will also receive resize consequences and may perform expensive internal layout or rendering. The manager must therefore minimize both its own paint work and unnecessary client configure traffic. [X11]

A reliable manager separates five forms of state:

1. **Intent** is the user or script request: split this leaf, focus left, set this ratio, map this client, enter fullscreen.
2. **Logical state** is the display-independent model: workspaces, layout trees, floating membership, focus target, fullscreen owner, and surface registry.
3. **Desired platform state** is the geometry, mapping, stacking, focus, and decoration state derived from the logical model.
4. **Applied platform state** is the manager's cache of what it has already asked X11 to do and what backing resources currently contain.
5. **Visible state** is the combination of manager-owned pixels and client-owned buffers that the user actually sees.

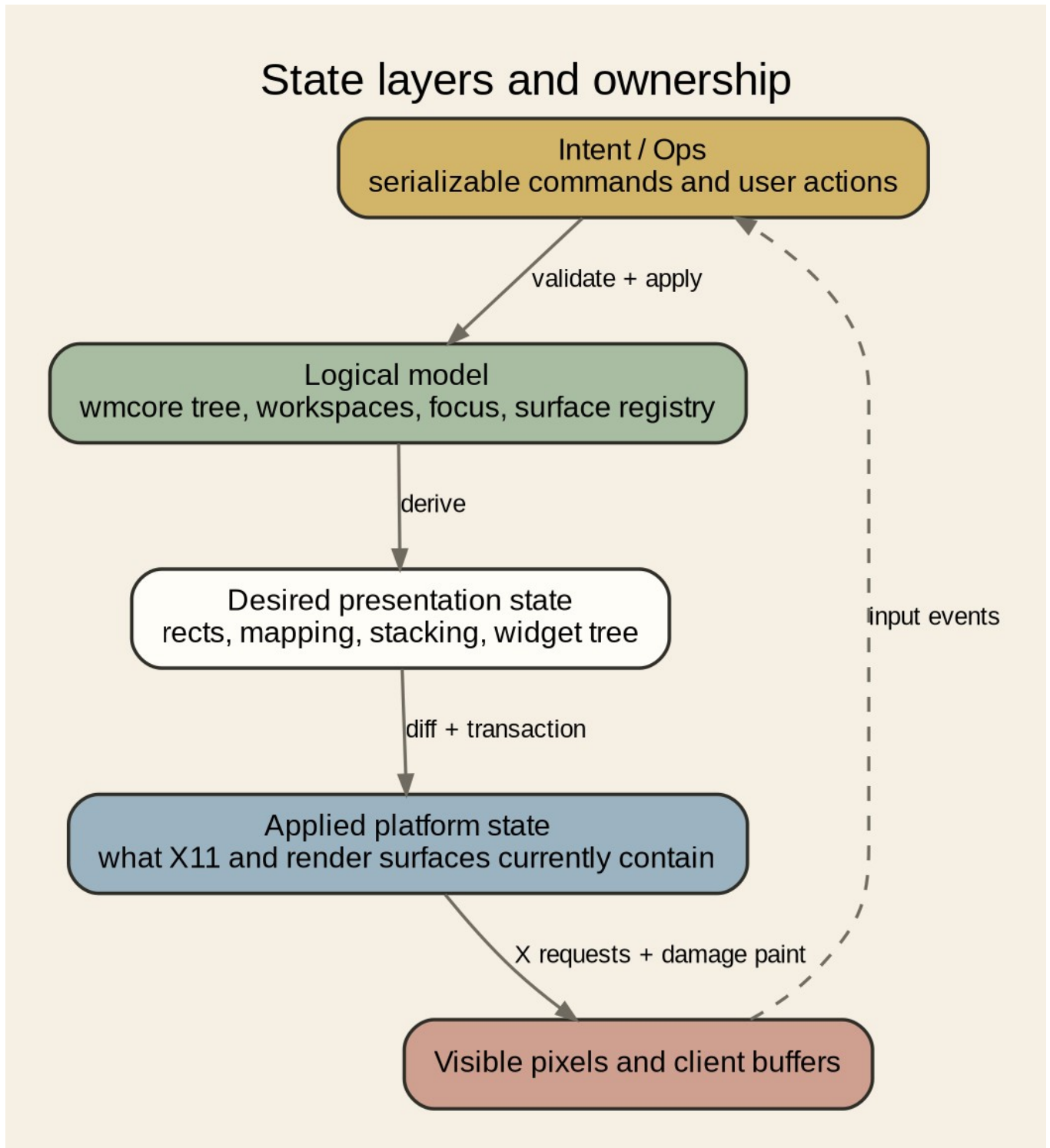


Figure 2. Intent should flow through a logical model into a desired/current platform-state diff.

When these layers are collapsed, unrelated work becomes coupled. A ratio update can accidentally cause every frame to map again, every divider to repaint, every bar to rasterize, and every builtin tile to rebuild. When the layers are explicit, a ratio update changes the logical tree, derives a small set of rectangle changes, sends only the necessary X requests, and repaints only surfaces whose pixels are invalid.

1.1 Reparenting and frame geometry

A reparenting manager creates an override-redirect frame window around each managed client and reparents the client as a child. The frame receives manager-owned input and decoration. The client receives an interior rectangle offset by the title strip and borders. Reparenting provides clear control over title strips, pointer gestures, and client stacking, but it introduces invariants:

- The manager must distinguish client windows from frame windows in every event handler.
- Destroy and unmap notifications can arrive on the client and can be caused by either the client or the manager.
- Client coordinates are relative to the frame, while EWMH and synthetic configure notifications often require root-relative coordinates.
- Resizing a frame and resizing its client are distinct requests.
- Server-side references such as a background pixmap must be detached before the resource is freed.
- The client's requested size hints apply to the client content rectangle, not the outer frame.

The current project documentation records several bugs from missing teardown steps and misdirected event registration. The developer guide's rule that destruction must detach callbacks, drop buffers, release server-side references, and clear all maps is correct and should be converted into one idempotent `FrameResources.Close()` path rather than maintained as a checklist in multiple callers. [GG-DEV]

1.2 ICCCM and EWMH are behavior, not metadata decoration

ICCCM defines the lower-level contract between clients and managers: `WM_NORMAL_HINTS`, `WM_TRANSIENT_FOR`, `WM_PROTOCOLS`, `WM_HINTS`, `WM_CLASS`, text properties, synthetic configure notifications, state transitions, and focus conventions. EWMH adds the desktop conventions expected by taskbars, pagers, launchers, and modern applications: active window, client lists and stacking, desktop assignment, window types, state requests such as fullscreen, close requests, work areas, moveresize requests, allowed actions, and more. EWMH explicitly builds on ICCCM rather than replacing it. [ICCCM] [EWMH]

The existing `ewmh.go` publishes the supporting-WM check, client list, active window, workspace count, current desktop, desktop names, and per-window desktop. That is a useful start. It should be treated as an incomplete compliance surface, not as a finished subsystem. The supported atom list should only claim behavior that is implemented. Each newly supported atom needs an event handler, state update, tests, and synthetic-client fixture where applicable.

A practical compliance sequence is:

Stage	Required behavior	Why it matters
Baseline ICCCM	<code>WM_PROTOCOLS/WM_DELETE_WINDOW</code> , <code>WM_NORMAL_HINTS</code> , <code>WM_TRANSIENT_FOR</code> , <code>WM_HINTS</code> input model, synthetic <code>ConfigureNotify</code> , clean Withdrawn/Normal transitions.	Prevents broken close behavior, incorrect focus, resize loops, and confused toolkits.
Desktop basics	<code>_NET_CLIENT_LIST</code> , <code>_NET_CLIENT_LIST_STACKING</code> , <code>_NET_ACTIVE_WINDOW</code> , desktop count/names/current, <code>_NET_WM_DESKTOP</code> .	Allows pagers and taskbars to represent the manager accurately.
Window classification	<code>_NET_WM_WINDOW_TYPE</code> and transient/group relationships.	Provides consistent float, dialog, utility, dock, and splash policy.
State control	<code>_NET_WM_STATE</code> , fullscreen, hidden, demands-attention, above/below where supported.	Allows clients and external tools to request or observe state without private IPC.
Work areas and docks	<code>_NET_WORKAREA</code> , desktop geometry/viewport, struts and partial struts.	Required before script-defined bars and taskbars can coexist with external docks or multiple outputs.
User operations	<code>_NET_CLOSE_WINDOW</code> , <code>_NET_MOVERESIZE_WINDOW</code> , <code>_NET_WM_MOVERESIZE</code> , <code>_NET_RESTACK_WINDOW</code> , <code>_NET_WM_ALLOWED_ACTIONS</code> .	Makes behavior interoperable with panels, accessibility tools, and automation.

1.3 Focus is a state machine

Focus is not a property that can be inferred locally from the most recent click. A tiling manager must account for the active workspace, focused leaf, floating focus, fullscreen ownership, modal/transient relationships, focus history, pointer-enter policy, and the possibility that a client refuses or redirects focus through ICCCM protocols.

The current `focusState` and `fullscreenState` refactor is a strong improvement. It turns a coupled set of fields and conventions into explicit ownership. The comments document a key invariant: exactly one of a tiled leaf, a floating client, or a

fullscreen client owns keyboard focus, while the tiled focus beneath a float can remain available for restoration. That model should become the template for other shell state such as grabs, modal focus scopes, pointer drag modes, and active accept contexts. [GG-FOCUS]

The next step is to distinguish **logical focus** from **X input focus** and **surface focus**. Logical focus identifies the selected desktop object. X input focus identifies the X window receiving key events. Surface focus identifies the widget node receiving text or navigation inside a manager-owned surface. A popup menu can own surface focus while the underlying client remains the logical selection; a modal dialog can own all three. Encoding those states separately prevents later menu and widget work from reintroducing the same class of coupled-field bugs.

2. A systematic method for building the manager

A new window-manager developer is exposed to many features at once: layout, focus, key bindings, EWMH, drawing, launchers, scripting, and multiple monitors. The correct development order follows invariants and observability rather than visible feature appeal.

2.1 Start from an executable state model

The manager should have one authoritative logical state and one mutation vocabulary. go-go-wm already does this for layout through `wmcore.Op`. Preserve that discipline as new features arrive. A user gesture, a key binding, IPC, JavaScript, a replay file, and a test should all produce the same operation or command descriptor. Code that mutates the tree directly from one path creates a second semantics and eventually a replay or focus discrepancy.

Not every action belongs in the layout tree. Floating geometry, fullscreen ownership, pointer grabs, pending interactive transactions, output configuration, bars, popup surfaces, and runtime registrations are shell state around the tree. They still need explicit commands and state machines. The rule is not “put everything in `wmcore`.” The rule is “every mutation has one owner and one named path.”

A useful top-level command envelope is:

```
type Command struct {
    ID          CommandID
    Principal   PrincipalID
    Seat        SeatID
    Time        time.Time
    Payload     CommandPayload
    Cause       Cause // key, pointer, script, IPC, client-message, replay
}

type CommandResult struct {
    Events []Event
    Effects []PlatformEffect
    Undo   *Command
}
```

`wmcore.Op` can remain the payload for tree mutations. Other payloads can cover focus, floating state, output changes, surface lifecycle, runtime lifecycle, and capabilities. This envelope provides provenance and makes traces educational: a developer can see not only that a ratio changed, but which pointer sequence, script, or client request caused it.

2.2 Write invariants before handlers

Each subsystem should begin with a short invariant list and pure decision functions. The recent focus work demonstrates the value. Examples include:

- Every managed client has exactly one frame and exactly one ownership classification: tiled, floating, dock, or ignored.
- A tiled leaf contains at most one client or builtin/script app identity.
- Every visible managed frame belongs to the active workspace or a global shell layer.
- Exactly one logical focus target exists per seat.
- A fullscreen owner, when present, owns geometry and focus for its scope.
- Every X resource has one owner and an idempotent destruction path.
- A runtime may invoke only effects permitted by its capabilities.
- A widget event targets a node that exists in the same committed tree revision used for hit testing.
- An accept answer must belong to the active interaction scope and satisfy the requested type predicate.

These invariants should appear in tests and in debug-only assertions. State-machine bugs are easier to correct when a failing trace names the invariant rather than only an X error.

2.3 Build the platform adapter as a reconciler

The X11 shell should derive desired platform state and reconcile it against cached applied state. i3's `con_state` is an instructive implementation: it stores mapping, frame rectangle, client rectangle, names, and other state representing what X currently sees, then `x_push_changes` sends only differences. Its decoration renderer also caches render parameters and skips raster work when the title, colors, geometry inputs, and relevant flags are unchanged. [\[I3-X\]](#) [\[I3-DECO\]](#)

Sway's transaction machinery solves a different platform problem, but the architectural lesson transfers. Sway accumulates dirty nodes, copies pending state into transaction instructions, configures only views whose content geometry changed, waits or times out, and then applies a coherent current state. go-go-wm does not need Wayland configure acknowledgement machinery on X11. It does need dirty nodes, pending/current state, coalescing, and one commit boundary. [\[SWAY-TXN\]](#)

2.4 Make every expensive path measurable

A window manager is latency-sensitive software. CPU percentage alone is insufficient because short freezes and event backlog dominate perception. Each hot path should emit structured timing and count data behind a debug switch:

```
resize seq=184 pointer_ts=... commit_ts=... latency=7.8ms
coalesced=4 dirty_nodes=3 layout=72us reconcile=41us
x_configures=4 maps=0 paints=0 upload_bytes=0 client_resizes=2
```

The minimum measurements are:

- event arrival to geometry request;
- geometry request to flush;
- accepted, coalesced, and dropped pointer samples;
- logical operations and dirty nodes;
- layout time and tree lookup count;
- X request count by type and bytes where available;
- manager paint time by surface and damaged pixels;
- upload time and bytes;
- client configure count;
- event queue depth and runtime callback latency;
- garbage collections and allocated bytes during a scripted interaction.

The existing pprof work provides valuable CPU evidence. Add `runtime/trace`, pprof labels by surface and operation, and an internal trace ring that the PBUI inspector can display. The instrumentation should eventually feed a script-defined performance workbench built with the same table, plot, timeline, and presentation widgets used by the rest of the environment.

2.5 Grow features through vertical slices

A feature is complete only when its model, platform behavior, scripting surface, events, tests, and inspection exist together. A practical vertical-slice checklist is:

6. Define state and invariants in the lowest appropriate package.
7. Define a command or operation and deterministic result.
8. Implement the X11 effect or broker effect on the owning loop.
9. Emit canonical events with cause and revision.
10. Add IPC and JavaScript adapters that invoke the same command.
11. Add an inspector view and trace fields.
12. Add unit, property, protocol, and end-to-end tests.
13. Add an example script that remains a smoke fixture.
14. Document resource ownership and teardown.
15. Benchmark the interaction when it can occur on a hot path.

This method is slower for the first visible demo and faster for every subsequent feature because it prevents special paths from accumulating.

Part II. Review of the current implementation

3. What the three-day sequence established

The repository archive from 18-20 July records a compressed but coherent design progression. Reading the tickets as one sequence is more useful than reading them as isolated features because each ticket exposes a new boundary.

Ticket	Primary contribution	Architectural boundary exposed	Long-term implication
GGWM-002	Goja scripting DSL with wm and pbui modules and multiple attachment points.	JavaScript owner loop versus WM and broker loops.	Scripts must remain clients of validated commands, not direct owners of mutable Go state.
GGWM-003	JS-defined UI surfaces using normalized row/segment specifications.	Runtime execution versus render-time snapshots.	The data boundary should become a retained widget protocol, not be removed in favor of callbacks during paint.
GGWM-004	Themes and an i3-derived JavaScript configuration.	Policy in script versus mechanism in Go.	Key bindings, rules, layouts, and shell composition belong in inspectable data and commands.
GGWM-005	Profiling, faster fills, faster conversion, input throttling, and batch work.	Geometry, painting, upload, and garbage collection as separate costs.	Further gains require changing invalidation and surface ownership, not only optimizing loops.
GGWM-006	MIT-SHM shared pixmaps and batch operation support.	CPU raster work versus transport/upload work.	Shared memory is an optimization below the retained rendering model, not the rendering model itself.
GGWM-007	Floating transient and dialog layer outside the tiling tree.	Layout tree versus shell overlay state.	Popups, modals, bars, notifications, and taskbars also need explicit shell layers and policies.
GGWM-008	Unified launcher registry and frame keyboard substrate.	Command identity versus presentation surface.	Launcher entries should become one view over a general command graph.
GGWM-009	Rich PBUI REPL with live result presentations and derived views.	Evaluation runtime versus persistent, inspectable values.	The REPL is the natural control plane for runtime, object, command, and event inspection.
GGWM-010	Pull-request review, system primer, security and correctness fixes.	Feature growth versus invariant ownership and teardown.	New shell state should be encapsulated as explicit state machines before more features depend on it.
GGWM-011	Focus/fullscreen state encapsulation and subsequent implementation.	Coupled fields versus one authoritative owner.	Apply the same pattern to modal focus, pointer grabs, runtime state, accept scopes, and surface lifecycle.

The sequence also establishes a project-specific engineering style worth retaining:

- Pure and deterministic core operations are preferred over hidden mutation.
- Event loops own mutable worlds; crossings post closures or messages.
- JavaScript exports data across a plain boundary rather than Goja values or Go pointers.
- Errors in user handlers become visible events rather than process crashes.
- Examples serve as both teaching material and smoke fixtures.
- Performance work is grounded in profiles and before/after traces.

The attached PBUI shell prototype expresses the conceptual center directly. Every visible object is a typed presentation; commands accept objects instead of reparsing labels; type-directed actions are available from object menus; tiles and workspaces are themselves presentations; and the accept context is desktop-wide. The basketball prototype shows why this matters for serious applications rather than only a color demo: a player selected from a table, scatter plot, radar legend,

watchlist, or result can drive other views without those views knowing each other's implementation. The reusable idea is typed object identity and shared interaction context, not React or a specific visual style.

4. Package architecture and ownership review

The current package layering is mostly sound. The repository's own developer guide states the intended direction: lower layers do not import higher layers, `wmcore` is pure, `draw` owns software rendering, `xshm` owns shared upload surfaces, `wmx11` owns X11 state, and `jsmod` exposes native modules. [GG-DEV]

4.1 `wmcore`: preserve the pure model, add indexes around it

The split tree is a compact and understandable model. Leaves carry application identity, split nodes carry direction and ratio, and operations return serializable results. This is an appropriate core for a tiling manager. Keeping floating windows and fullscreen outside the tree is also defensible: these are shell modes that temporarily override or coexist with tiling geometry rather than persistent split structure.

The current implementation's main scaling issue is repeated traversal, not the tree representation itself. Layout is linear in the number of nodes, but reconciliation performs additional `Find` operations for frames and dividers. A ratio update can therefore perform several complete or partial traversals before X requests begin. With a small tree this is not the largest cost, yet it complicates dirty-subtree work and makes performance degrade less predictably.

Do not immediately replace the tree with a mutable parent-pointer structure. That would sacrifice some of the pure model's clarity. Introduce derived indexes at transaction boundaries:

```
type LayoutSnapshot struct {
    Revision    uint64
    WorkspaceID string
    ByNode      map[NodeID]NodeSnapshot
    RectByNode  map[NodeID]Rect
    Parent      map[NodeID]NodeID
    Leaves     []NodeID
    Splits      []NodeID
}

type NodeSnapshot struct {
    Kind  NodeKind
    App   string
    Dir   Direction
    Ratio float64
}
```

A snapshot is rebuilt once after a committed logical mutation or incrementally for a dirty subtree later. It allows focus navigation, divider reconciliation, frame reconciliation, scripting queries, and debugging to share one indexed view. It also gives traces a model revision so stale input can be detected.

A second improvement is an affected-set result from operations. `SetRatio(split)` should report that the split subtree is geometrically dirty. `SetLeafApp(leaf)` should report content and decoration dirtiness for one leaf but no geometry change. `SwapLeaves` should report two content identities and perhaps focus/decoration changes without forcing unrelated layout work. The core need not know X11; it only needs to classify logical invalidation.

```
type DirtySet struct {
    GeometryRoots []NodeID
    Content        []NodeID
    Decoration     []NodeID
    Focus         bool
    WorkspaceMeta bool
}
```

This classification becomes the bridge from pure operations to efficient platform reconciliation.

4.2 `wmx11`: strong ownership, overly broad reconciliation

The X11 shell correctly centralizes state on one loop. `ScriptBackend` posts to that loop and waits with context bounds. Key callbacks are expected to post rather than run JavaScript or mutate state directly. This is the correct concurrency contract. [GG-SCRIPTING]

The shell currently mixes several responsibilities inside frame and relayout paths:

- deriving desired rectangles from the tree;
- creating, moving, resizing, mapping, and unmapping X windows;
- configuring client interiors;

- maintaining fullscreen and floating exceptions;
- painting title strips, borders, builtin content, and scripted content;
- creating or resizing upload resources;
- publishing events and EWMH state.

These responsibilities can remain in `wmx11` as a package, but they should be split into components with explicit caches:

WM loop	
└─ ModelController	applies commands and owns logical shell state
└─ LayoutProjector	builds LayoutSnapshot and DirtySet
└─ XReconciler	desired/current geometry, map, stack, focus
└─ ChromeManager	title/border/divider/bar resources
└─ SurfaceCompositor	builtin/script snapshots, damage, upload
└─ ProtocolManager	ICCCM/EWMH properties and client messages
└─ InputRouter	grabs, drags, focus scopes, widget events

This split is not intended to create asynchronous rendering immediately. The components can run on the same owner loop. Their purpose is to make it impossible for a geometry-only change to call a content renderer accidentally.

4.3 Focus and fullscreen: a model to copy

The implemented `focusState` and `fullscreenState` deserve explicit approval. The refactor introduces named queries such as `OwnsGeometry`, `OwnsFocus`, `FocusTarget`, `FocusedLeaf`, and `FocusedFloat`, and it preserves a tiled focus target beneath floating or fullscreen state. The comments explain the invariant and the old failure modes. This is significantly safer than scattered reads of `w.fullscreen`, `w.focused`, and `w.focusedFloat`. [GG-FOCUS]

Use the same shape for:

- `interactionState`, which owns pointer drag, divider resize, move/resize mode, active button, and cancellation;
- `modalState`, which owns the modal surface stack and restoration target;
- `acceptState`, which owns active scoped accepts per seat or runtime;
- `surfaceFocusState`, which owns focused widget node and focus traversal within manager surfaces;
- `runtimeRegistry`, which owns runtime identities, lifecycle states, capabilities, queues, and exported resources.

Each state object should own both reads and writes, expose pure decision helpers, and provide a small set of state transitions. This prevents future shell features from becoming another group of fields connected only by convention.

4.4 `draw` and `xshm`: useful primitives at the wrong granularity

The optimized fill and conversion loops, cached images, and MIT-SHM surfaces are valuable. The repository's profiles show that the first performance pass substantially reduced CPU time. The remaining issue is that the surface being optimized is frequently the wrong size. A title strip and border are narrow decorations, but the current frame image can cover the entire pane, including client-sized areas. Resizing a pane changes that large image's dimensions, so the manager must allocate or recreate resources and touch many pixels even when the visible title text and colors did not change.

MIT-SHM should remain an upload backend. Its API should evolve to accept damaged rectangles and long-lived surface allocations. The rendering layer should decide whether a surface needs repaint and what region changed. `xshm` should not be asked to infer damage from full RGBA replacement.

Three resource strategies are useful:

16. **Fixed chrome surfaces.** Title strips, buttons, and dividers use narrow buffers whose height or width is stable across ordinary pane resize.
17. **Retained application surfaces.** Builtin and scripted widgets retain their last complete surface. During interactive resize, the manager can clip, letterbox, or temporarily scale/anchor it, then rerender at release or at a lower budget.
18. **Bucketed reusable buffers.** When a variable-size software surface is necessary, allocate from size classes or reuse capacity rather than destroy and recreate on every pixel change. The valid rectangle and stride can differ from capacity.

4.5 PBUI core and broker: the correct seam, prototype identity semantics

The `pbui` package is correctly independent of I/O. `Object` is an open presentation type plus JSON value, label, and documentation. `Verb` is data. The broker owns accept state, verb registration, and events without linking X11. These boundaries are appropriate for cross-process composition. [GG-OBJECT] [GG-BROKER]

The current identity model is sufficient for scalar demos but not for a persistent desktop environment. Two objects with equal JSON payloads are not always the same live object, and a large or mutable object should not be copied through every menu request. Introduce references and providers:

```

type ObjectRef struct {
  PType      string    `json:"ptype"`
  ID         string    `json:"id"`
  Revision   uint64    `json:"rev,omitempty"`
  Provider   PrincipalID `json:"provider"`
  Label      string    `json:"label,omitempty"`
  Doc        string    `json:"doc,omitempty"`
  Snapshot   json.RawMessage `json:"snapshot,omitempty"`
}

```

The provider answers **resolve**, **describe**, **present**, **serialize**, and perhaps **subscribe** requests. The snapshot is an optional cached face, not the identity. A file object can remain valid after its label changes; a window object can resolve to current geometry; a REPL result can preserve a value in its runtime; a trace event can be immutable and globally serializable.

The broker currently uses a client-supplied name as verb ownership and keeps one global accept session. Both should change before the broker becomes an OS substrate.

- A connection receives a generated **PrincipalID**. A human-readable name is metadata and may be duplicated.
- Verb IDs are namespaced by principal or package, with explicit collision rules.
- Disconnect cleanup uses the principal, never a display name.
- Accept sessions have explicit owner, seat, scope, parent session, and cancellation policy.
- Control replies and interaction results are reliable. Best-effort events use a separate queue or subscription stream.
- The handshake includes protocol version, authenticated peer information where available, requested roles, and granted capabilities.
- Event sequence numbers are per canonical stream and can expose gaps to subscribers.

A simple reliability classification is:

Message class	Delivery policy	Slow-consumer behavior
Request reply, accept result, capability decision, lifecycle acknowledgement	Reliable and ordered.	Apply backpressure within a bounded timeout; disconnect or fail the request rather than silently drop.
State notification such as focus, pointer position, hover text, resize preview	Latest value wins.	Coalesce by key and replace pending state.
Canonical operation and lifecycle event	Ordered with gap detection.	Persist briefly or disconnect the subscriber when it cannot keep up.
Telemetry, paint samples, pointer traces	Best effort or sampled.	Drop with counters and adaptive sampling.

4.6 **j_{smod}**: disciplined loop crossings, incomplete runtime isolation

The current JavaScript bridge contains several good constraints. Goja runs on its owner event loop. Broker work occurs off the VM thread. Promise settlement is posted back. Event delivery is bounded. Render hosts consume normalized snapshots. Values crossing from the VM are exported into plain structures. These constraints should be preserved as the API grows. [GG-JSBRIDGE] [GG-UI]

The next risks are runtime-global state and synchronous work on the owner loop. Some **pbui** calls perform bounded socket round trips synchronously. A two-second bound prevents permanent deadlock but can still freeze every timer, handler, and UI update in that runtime. Convert control-plane APIs to promises where they can block, or move the blocking operation to a worker and post completion. Reserve synchronous methods for local validation and cached queries.

Event queues need semantics, not one generic overflow rule. A queue that drops new items can preserve stale focus or stale resize events while discarding the latest state. A queue that drops oldest items can lose the beginning of a command sequence. Define an event class at publication and use coalescing keys or reliable channels accordingly.

The xgoja provider interface already offers a useful hook: create module factories per runtime and pass an owner, context, and closer. Use that to ensure provider state is runtime-local. A module factory must not accidentally share verb handlers, UI app registries, or subscriptions across runtimes unless the shared object is an explicit broker client with its own synchronization and identity.

4.7 **ui** and **uispec**: a good serialization boundary, not yet a widget system

The current **ui.app** contract demonstrates the right safety property. JavaScript produces a spec; Go validates and normalizes it; X rendering reads only the latest snapshot; clicks and keys post back to the runtime; a throwing render preserves the previous valid frame and emits a visible error. That is a robust foundation. [GG-UI-DOC]

The current flat row/segment model has deliberate limits:

- It has no stable nested identity beyond action names and row position.
- Every handler reruns the whole render function and replaces the complete snapshot.
- Layout is simple packing rather than a general constraint or intrinsic-size system.
- Hit regions are regenerated for the full surface.
- There is no distinction between content redraw and layout invalidation.
- There are no reusable stateful components, virtualized lists, scroll models, focus scopes, accessibility semantics, or input-method integration.
- Surface role, placement, stacking, keyboard policy, and lifetime are handled outside the widget description.

Do not discard the flat IR. Treat it as Widget Protocol v0 and create an explicit v1 retained tree while maintaining compatibility through a translator. The attached basketball prototype supplies a practical widget inventory: sortable tables with inline bars, linked focus, scatter plots, shot charts, radar charts, line charts, watchlists, inspectors, and traces. These are not niche sports widgets. They are exactly the views needed for a developer-oriented desktop: process tables, latency plots, event timelines, allocation charts, dependency graphs, object watchlists, and trace inspectors.

4.8 The rich REPL: the strongest product direction and the most important scheduler risk

The REPL session retains cells, raw values, derived views, console output, and PBUI presentations. It derives number, color, series, dataset, palette, and JSON views, supports `Out(n)` and `$.`, and allows values to provide a custom `__pbui__` representation. This is the correct direction. A result is not merely formatted text; it is a desktop object with multiple possible views and actions. [\[GG-REPL-DOC\]](#)

The main correctness issue is interruption. Evaluations are serialized, but an infinite loop or expensive synchronous computation can monopolize the runtime owner because the cell uses the session lifetime context rather than an evaluation deadline and explicit Goja interrupt. The go-go-goja engine already demonstrates a shutdown pattern that invokes `VM.Interrupt` after a grace period. Reuse that mechanism for cells, with care around runtime consistency.

A cell should have an explicit state machine:

```

queued -> compiling -> running -> settling-promises -> complete
      |               |               |
      +-> cancelling -+
      +-> timed-out
      +-> interrupted -> runtime-healthy | runtime-restart-required

```

The input path also needs protection. A key handler must never block the X/WM loop while sending to a full evaluation queue. Use a non-blocking enqueue and render a visible “queue full” or “busy” state. Text editing must be rune-aware, and the long-term editor needs cursor motion, selection, clipboard, multiline input, history search, completion, bracket matching, and IME support.

5. Current strengths to preserve

A redesign should not erase the project’s differentiators. Preserve the following properties explicitly.

One mutation vocabulary

Keyboard, mouse, IPC, JavaScript, tests, and replay should continue to share operations. The fluent JavaScript API is sugar over normalized operations, not a second mutable object model.

JavaScript outside render paths

No X expose, paint, hit-test, or geometry commit should call JavaScript. JavaScript may produce snapshots and effects on its owner loop. The manager commits those snapshots independently.

Typed objects across application boundaries

Applications should compose through presentation types and commands, not by importing one another or knowing concrete component instances. A color lab can accept a color from a note, a plot, a REPL cell, or a file metadata inspector because the type contract is shared.

Errors as inspectable objects

Script errors should remain events and should become richer presentations: error, stack frame, source location, runtime, capability denial, invalid widget path, and failed command. Errors should be clickable and inspectable rather than only logged.

Pure models and state-machine tests

`wmcore`, focus decisions, fullscreen decisions, floating classification, layout plans, rule matching, and widget layout should remain testable without an X server. Xvfb fixtures should test the adapter, not compensate for avoidable logic in event handlers.

Historical ideas expressed in modern boundaries

CLIM presentation types associate objects with visual output and allow commands to request typed arguments that can be satisfied by clicking compatible visible presentations. HyperCard's message path routes an event through a defined object hierarchy when a specific object does not handle it. Live Smalltalk environments integrate workspaces, inspectors, browsers, debuggers, and the running object system. The transferable ideas are typed interaction, structured propagation, and live inspectability. The implementation should use explicit identities, immutable snapshots, capabilities, and supervised runtimes rather than copying historical global-state assumptions. [\[CLIM\]](#) [\[HYPERCARD\]](#) [\[SMALLTALK\]](#)

Part III. Resize performance and rendering architecture

6. Trace the current divider resize before changing it

The divider path begins with pointer motion on an input window. The handler rejects samples that arrive within roughly 16 ms of the previous accepted sample. For an accepted sample, it obtains layout geometry, locates the split, computes a clamped and snapped ratio, applies a `set-ratio` operation, and calls the resize-specific relayout path. The relayout derives workspace geometry, synchronizes divider windows, reconciles visible frames, configures changed clients, and paints changed manager-owned surfaces. [GG-INPUT] [GG-MANAGE]

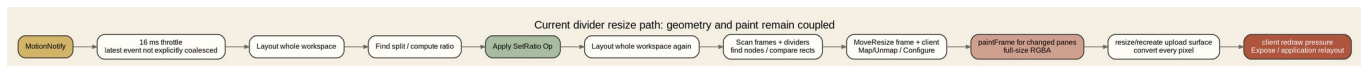


Figure 3. Current interactive resize path. The throttle reduces frequency but leaves geometry and full paint coupled.

The path contains several distinct costs. They should be measured separately because each requires a different intervention.

6.1 Input scheduling cost

A timestamp throttle ignores events that arrive too soon. It does not necessarily retain the newest ignored pointer position for the next frame. Depending on event delivery and handler timing, the visible divider can lag behind the pointer or advance in irregular steps. A latest-only coalescer is different: every motion updates one pending coordinate, and a scheduler consumes the newest coordinate at the next commit opportunity. Intermediate coordinates are intentionally discarded while the final coordinate is preserved.

The coalescer should be owned by `interactionState`:

```
type DividerDrag struct {
    Split      wmc core.NodeID
    Axis       Axis
    StartRatio float64
    PendingPos  int
    PendingSeq  uint64
    Scheduled   bool
    LastCommit  time.Time
}

func (s *interactionState) OnMotion(pos int, seq uint64) {
    s.drag.PendingPos = pos
    s.drag.PendingSeq = seq
    if !s.drag.Scheduled {
        s.drag.Scheduled = true
        s.scheduler.Request(FramePriorityGeometry, s.commitDividerSample)
    }
}
```

The commit callback clears `Scheduled`, reads the latest position, applies one preview transaction, and schedules another commit only if a newer sample arrived while it ran. This produces bounded work with no stale final position.

6.2 Logical layout cost

`wmc core.Layout` is not intrinsically expensive for a small binary tree. The problem is repeated derivation and lookup around it. The resize handler and relayout path can each traverse the tree, and reconciliation can recursively find nodes again for frames and dividers. The first optimization is not a clever layout algorithm. It is one indexed snapshot per logical revision.

During a divider drag, only the selected split subtree changes rectangles. Ancestors preserve their outer rectangles; nodes outside the subtree are unchanged. A future incremental layout function can take the selected split's already-known rectangle and produce rectangles only for descendants:

```
func LayoutSubtree(n *Node, outer Rect, out map[NodeID]Rect) {
    out[n.ID] = outer
    if n.Kind == Leaf { return }
    a, b := divide(outer, n.Dir, n.Ratio)
    LayoutSubtree(n.A, a, out)
    LayoutSubtree(n.B, b, out)
}
```

The initial implementation can still run full **Layout** once per accepted transaction while eliminating repeated **Find** calls. That change is low risk and creates the data structures needed for incremental layout later.

6.3 X request cost

A changed split normally changes the rectangles of leaves in both child subtrees and the divider between them. The manager must move/resize frame windows and client interiors. It should not map already mapped windows, reassert unchanged stacking, reset unchanged properties, or repaint unrelated dividers.

Maintain applied state per X-owned object:

```
type AppliedFrameState struct {
    FrameRect Rect
    ClientRect Rect
    Mapped     bool
    StackLayer Layer
    BorderState BorderVisualState
    TitleKey   TitleRenderKey
}
```

Reconciliation compares desired and applied states and records effects:

```
if desired.FrameRect != current.FrameRect {
    effects.ConfigureFrame(f, desired.FrameRect)
}
if desired.ClientRect != current.ClientRect {
    effects.ConfigureClient(f.client, desired.ClientRect)
}
if desired.Mapped != current.Mapped {
    effects.SetMapped(f, desired.Mapped)
}
```

The effects can be sent as a batch of unchecked XCB requests followed by one flush, with checked requests reserved for setup paths or debug mode. The applied cache updates when requests are issued because X11 does not provide a normal configure-ack transaction. X errors should still be associated with object and revision in debug builds.

6.4 Manager paint cost

This is the dominant architectural cost. **paintFrame** can allocate or resize a pane-sized RGBA image, fill the full background, render manager-owned content, convert pixels, and update an X surface. During resize, dimensions change continuously, so caches keyed by exact width and height miss continuously. Shared memory reduces the transfer path but every pixel is still written. The existing profile documents show that full fills, RGBA conversion, expose repaint, and garbage collection were major contributors before the first optimization pass. [\[GG-PERF\]](#) [\[GG-XSHM\]](#)

The correct question is not “How can full-frame paint become another 20 percent faster?” It is “Which visible pixels actually belong to the manager, and which of those pixels changed?” For a normal client frame, the manager owns a title strip, a few border rectangles, and perhaps title buttons. The client owns the large interior. A divider resize changes the position and extent of those rectangles, but not necessarily title text, colors, icons, or button state.

6.5 Client redraw cost

Even a zero-cost manager configure can trigger expensive application work. Terminal emulators reflow grids, browsers relayout documents, IDEs recalculate panes, and GPU clients may recreate swapchains. Sending duplicate or overly frequent sizes creates client-side latency outside the manager's profile.

The manager should therefore expose interactive resize policies:

- **live** configures clients at each geometry commit.
- **decorations** moves manager chrome and a preview while sending client size less frequently.
- **outline** shows only a divider or rectangle and configures clients once on release.
- **adaptive** begins live, detects missed frame budgets or slow clients, and lowers configure frequency.

live should remain the default target because it feels direct when the system is fast. The other modes are useful diagnostic and compatibility tools. They also make it possible to isolate manager cost from client cost in benchmarks.

7. The target geometry transaction

Interactive resizing should run through a dedicated geometry transaction rather than the full general relayout-and-paint path.

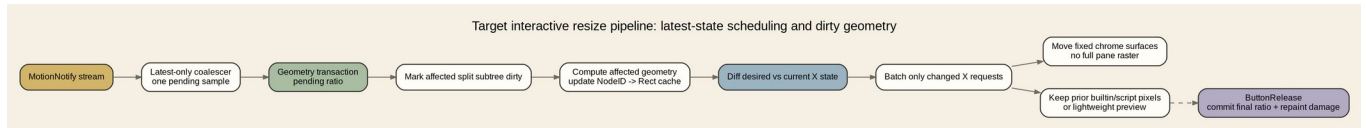


Figure 4. Target path. Pointer samples update pending state; only the latest sample becomes a dirty-geometry transaction.

7.1 Transaction stages

A geometry transaction has six stages:

19. **Capture intent.** Store the latest pointer position and derive a preview ratio without mutating unrelated shell state.
20. **Apply logical preview.** Update a pending tree revision or a temporary ratio override for the selected split.
21. **Derive dirty geometry.** Recompute rectangles for the affected subtree and divider.
22. **Diff platform state.** Compare desired frame/client/divider rectangles with applied X state.
23. **Commit X effects.** Send only changed configure requests and flush once.
24. **Schedule paint.** Mark chrome or application surfaces damaged according to policy. Ordinary client chrome may require no raster. Builtin/script surfaces may retain old pixels until a budgeted repaint or release.

The final button release commits the ratio as a canonical `wmcore.Op`, emits the durable operation event, repaints at final dimensions, and clears preview state. Preview samples may emit sampled telemetry but should not flood the canonical operation log unless continuous ratio history is a deliberate feature.

This distinction also improves undo and replay. One drag becomes one logical operation with start and end ratios, not sixty durable ratio operations.

7.2 Preview state versus committed state

There are two implementation choices.

Temporary tree mutation

Apply `SetRatio` to the logical tree for every preview and suppress durable event logging until release. This reuses existing layout code but can expose transient revisions to scripts and queries. It also makes cancellation require restoring the start ratio.

Ratio override layer

Keep the committed tree unchanged and store `previewRatios[splitID]`. Layout reads the override when present. Release applies one operation; cancellation drops the override. This keeps scripts and persistence on committed state while the platform shows a preview. It is the cleaner long-term design.

```

type LayoutInputs struct {
    Root      *wmcore.Node
    RatioOverride map[wmcore.NodeID]float64
}
  
```

Queries can optionally request `state: "committed" | "presented"` when there is value in inspecting the preview.

7.3 Dirty propagation

Dirty propagation should distinguish at least:

- **geometry dirty**, when rectangles or placement change;
- **layout dirty**, when a widget subtree needs measurement and arrangement;
- **paint dirty**, when pixels change but geometry remains valid;
- **hit-test dirty**, when interactive regions change;
- **semantics dirty**, when accessibility or inspection metadata changes;
- **protocol dirty**, when EWMH/ICCCM properties change;
- **stacking dirty**, when layer order changes.

A divider preview marks geometry for one split subtree and its divider. It does not mark title text, theme, workspace metadata, bars, launcher contents, or unrelated app surfaces dirty. A title property change marks one title strip paint dirty without relayout. A theme change marks many paint nodes dirty and may mark layout dirty only if metrics such as title height or padding change.

AwesomeWM's widget protocol explicitly distinguishes `layout_changed` from `redraw_needed`. go-go-wm should adopt the same conceptual separation throughout both frame chrome and script widgets. [AWESOME-WIDGET]

8. Separate chrome from pane-sized surfaces

The highest-value implementation change is to stop representing a client frame's decoration as a pane-sized image.

8.1 Recommended X window structure

A client frame can use this hierarchy:

```
frame window (manager-owned container; no large painted background)
├── title window      fixed height, variable width
├── client window      reparented application
├── optional border    input windows or X border
└── optional resize handles (InputOnly)
```

The frame itself establishes containment and clipping. The title window owns a narrow backing surface. Borders can be the frame's X border where one uniform color is sufficient, four narrow child windows when per-edge visuals or input are needed, or Shape/XFixes regions for more advanced decoration. The client fills the interior and can be resized independently.

When pane width changes, the title window changes width. That may still require a surface resize, but its pixel count is `width * titleHeight`, not `width * paneHeight`. When pane height changes, title pixels need not change at all. Border windows move or resize using X geometry, often without raster work.

8.2 Title render keys

A title strip should rerender only when a render key changes:

```
type TitleRenderKey struct {
    Width      int
    Title      string
    IconRevision uint64
    Focused    bool
    Urgent     bool
    ThemeRevision uint64
    ButtonState uint32
}
```

If only frame height changes, the key is unchanged. If width changes but the prior rendered title remains valid inside the new width, a later optimization can preserve most pixels and fill or clip the edge. Start with exact-width rerender because the surface is narrow; the major gain comes from not repainting the pane interior.

8.3 Builtin and script tiles

Builtin/script tiles are different because the manager owns their full content. They need retained application surfaces and resize policy.

During a drag, choose one of four behaviors per app or global setting:

25. **Immediate rerender.** Use for simple surfaces with measured render cost below budget.
26. **Rate-limited rerender.** Render at 15-30 Hz while geometry commits at 60-120 Hz.
27. **Retained clipping.** Keep the previous surface anchored at the top-left and clip or expose a background in newly visible regions.
28. **Preview transform.** Scale the prior surface for visual continuity, then rerender sharply on release. This is easier in a compositor or XRender path than a plain software/XCopyArea path and can be deferred.

The default should be cost-aware. Each app renderer can maintain an exponential moving average of render plus upload time. The scheduler grants a surface a live-resize budget only when it has historically completed within that budget.

8.4 Dividers and bars

Dividers have a very small visual state space: orientation, idle/hot/dragging/snapped, theme revision, and perhaps length. Cache reusable pixmaps for the repeated center glyph or draw the divider with X primitives. Do not allocate an image and repaint every divider on every workspace relayout. Move the divider window; repaint only when its visual state or size requires it.

Bars should use retained widget surfaces and damage. A workspace label changing focus should damage the old and new chips, not rasterize the whole screen-width bar. Expose events should copy the affected region from retained backing storage. If the backing store is missing, repaint only the exposed widget subtree and region.

9. A concrete implementation plan for fast resize

The plan below is ordered to produce measurable gains without requiring the final widget system first.

Phase R0: establish the benchmark and trace contract

Add a `perftrace` package with cheap no-op behavior when disabled. Assign every pointer drag a `DragID` and every preview commit a sequence. Record timestamps and counts at input, transaction begin, layout end, X flush, paint end, and release.

Add counters to the current path before changing it:

```
type ResizeSample struct {
    DragID      uint64
    Seq         uint64
    InputToFlush time.Duration
    LayoutTime  time.Duration
    ReconcileTime time.Duration
    PaintTime   time.Duration
    UploadTime  time.Duration
    NodesVisited int
    XConfigureCount int
    MapCount    int
    PaintPixels  int64
    UploadBytes  int64
    Coalesced   int
}
```

Expose a ring of recent samples as a PBUI `resize-trace` object with table, timeline, and summary views. This converts performance work into a self-observing desktop feature.

Phase R1: indexed snapshot and applied geometry cache

Build `LayoutSnapshot` once per relayout. Replace recursive lookups in frame and divider reconciliation with map access. Introduce `AppliedFrameState` and `AppliedDividerState`. Skip every duplicate map, unmap, configure, and paint decision.

This phase should not alter visible behavior. It establishes correctness tests around desired/current diffing.

Phase R2: latest-only drag coalescer and one-operation release

Replace timestamp-only throttling with pending latest position plus a geometry scheduler. Keep a ratio override during preview. Emit one canonical `set-ratio` operation on release. Add cancellation that restores committed geometry without another tree mutation.

Target criterion: pointer-to-X-flush p95 below one display frame under a terminal-only layout, with no unbounded event backlog.

Phase R3: paint suppression during client-only geometry changes

Add a resize path that configures frame and client geometry without calling full `paintFrame`. Preserve existing pixels until expose or release. Ensure title and border visuals remain valid enough for the experiment. This phase may show blank newly exposed frame regions in corner cases; use it to quantify the upper bound of paint removal before restructuring chrome.

Target criterion: manager paint pixels and upload bytes approach zero during terminal-only drag.

Phase R4: title child windows and border separation

Refactor frame creation and teardown to own title and border resources independently. Migrate title hit regions, drag gestures, menus, and focus visuals. Maintain an idempotent resource owner with tests for partial construction failure and repeated close.

Target criterion: terminal pane resize paints at most narrow title-strip damage, and frame height changes paint zero title pixels.

Phase R5: retained builtin/script surfaces and resize budgets

Wrap each manager-owned app in a `Surface` that retains its backing store, current size, damage, render cost, and revision. During resize, choose immediate or deferred repaint. On release, ensure a final exact-size render.

Target criterion: a complex data dashboard remains interactive because geometry commits continue even when its renderer is deferred.

Phase R6: dirty-subtree layout and batch effects

Use operation dirty sets to lay out only the affected split subtree. Batch X effects and flush once. Add assertions that no object outside the dirty subtree receives a geometry request.

Target criterion: work per sample scales with the affected subtree, not total workspace count or hidden workspace complexity.

Phase R7: partial damage and pooling

Add region-based painting, upload damaged rectangles, and reusable image/surface capacity. This phase improves bars, menus, rich widgets, and ordinary updates beyond resize.

Target criterion: changing one counter or focus chip touches a bounded region independent of surface area.

10. Performance test laboratory

A performance fix is complete only when it survives a repeatable matrix. The repository already uses Xvfb/Xephyr, xdotool, pprof, screenshot sampling, and an environment switch to disable SHM. Extend that harness rather than creating a separate benchmark culture.

10.1 Scenario matrix

Dimension	Values
Visible leaves	1, 2, 4, 8, 16, 32
Affected subtree	one leaf pair, half workspace, full workspace
Client type	xterm-like terminal, browser, GTK editor, fixed-size dialog, no client/builtin tile, complex script dashboard
Resize policy	live, rate-limited, outline, adaptive
Upload backend	SHM, PutImage fallback
Display	Xvfb, Xephyr, real Xorg/Xwayland session
Compositor	absent, present
Theme	each theme, plus large title/icon stress case
Runtime load	idle, event-heavy automation, slow widget handler, REPL evaluation

10.2 Metrics and budgets

Use latency distributions, not averages. Suggested initial budgets on ordinary local hardware are hypotheses to refine after measurement:

- Input-to-X-flush p95 under 8 ms for terminal-only layouts and under 16 ms with ordinary script widgets.
- No single manager-owned paint task over 8 ms on the WM loop; expensive surfaces are deferred.
- Zero full-frame RGBA allocations during a steady divider drag after warm-up.
- Zero map/unmap requests for windows whose visibility did not change.
- No repeated configure request when desired integer geometry is unchanged.
- One logical operation and one durable event per completed drag.
- Final pointer position applied within one scheduled commit after release.
- Event queues remain bounded and report coalescing or loss by class.

10.3 Correctness assertions during benchmark

Performance tests must also assert:

- no stale blank region remains after release;
- client and frame rectangles maintain title/border offsets;
- snapped ratios produce deterministic geometry;
- fullscreen cancels or blocks divider interaction consistently;
- workspace switching during a drag cancels or transfers the interaction according to one documented rule;

- destroying either affected client during a drag cleans up safely;
- a slow client cannot stall the WM loop;
- a slow script renderer cannot delay geometry commits;
- SHM disable/fallback produces identical final pixels and resource counts.

10.4 Diagnosing where latency remains

After the architecture change, remaining lag usually falls into one of four categories:

29. **WM loop occupancy.** A handler, synchronous socket request, paint task, or logging path blocks input processing.
30. **X server scheduling.** Too many requests, round trips, checked calls, or expensive server-side copies delay visible geometry.
31. **Client response.** The application relayouts slowly after configure.
32. **Compositor/display presentation.** The server applies geometry promptly but composition or frame presentation is delayed.

The trace should include enough timestamps and counters to separate these categories. A manager-only outline mode is especially useful: if outline motion is fast but live client resize is slow, manager paint is not the remaining bottleneck.

Part IV. PBUI and a fully scriptable surface runtime

11. Define PBUI as a protocol, not a component library

The presentation-based UI pattern has four independent concepts:

33. A **presentation object** has a type and identity.
34. A **renderer** gives that object one or more visible representations.
35. An **input context** requests an object satisfying a type and predicate.
36. A **command** declares that it applies to certain object types and may request more typed arguments.

The current code already implements recognizable versions of all four: `pbui.Object`, script or builtin renderers, the desktop-wide accept session, and type-directed verbs. The long-term design should make these concepts more explicit so they can support persistence, remote providers, richer type relations, multiple seats, and object inspection.

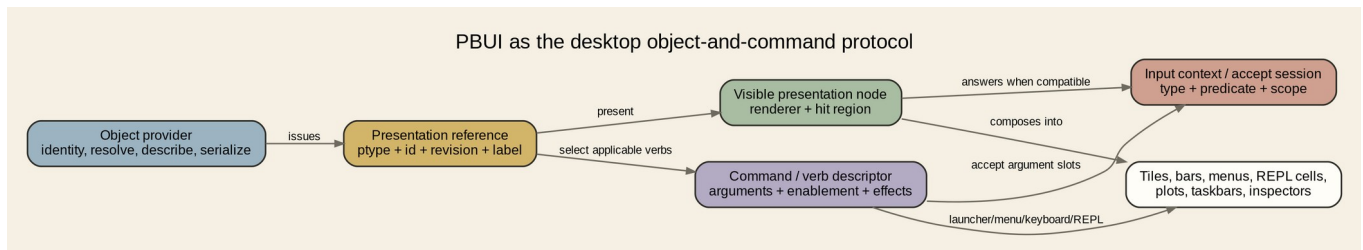


Figure 5. PBUI connects object providers, visible presentations, typed input contexts, and commands.

11.1 Type names and compatibility

Open string presentation types are a practical choice. They let packages introduce `git-commit`, `file`, `window`, `repl-cell`, or domain objects without modifying a central enumeration. Exact matching plus `any` is enough for the first protocol but becomes restrictive as generic tools appear. A file inspector should apply to `source-file`, `image-file`, and `directory-entry`; a revision command may accept both `git-commit` and another VCS revision.

Add an optional type registry with declared parents and schemas:

```

pbui.type({
  name: "git.commit",
  parents: ["vcs.revision", "inspectable"],
  schema: {
    type: "object",
    required: ["repo", "oid"],
    properties: {
      repo: { type: "string" },
      oid: { type: "string" }
    }
  }
});

```

Compatibility should be deterministic and side-effect free. A type is compatible when it is equal to the requested type or has that type in its registered ancestor closure. Predicates can further restrict values. Do not make arbitrary JavaScript predicates run inside the global broker. A requesting runtime can provide a serializable predicate supported by the provider or receive candidates and validate them locally before final acceptance.

11.2 Identity, snapshot, and liveness

A presentation should carry a reference, not assume the JSON payload is the entire object. Three categories matter:

- **Value objects** are self-contained and immutable: numbers, colors, immutable trace events, hashes.
- **Entity references** have stable identity and changing state: windows, files, processes, workspaces, runtimes, widgets.
- **Ephemeral references** are valid only while a provider or session remains alive: a raw REPL closure, a temporary selection, a debugger context.

The object reference should state lifetime and serialization behavior. A provider can expose:

```
object.resolve(ref)      -> current snapshot or gone
object.describe(ref)    -> structured inspector sections
object.views(ref, ctx)  -> render specifications
object.commands(ref)    -> applicable command IDs / enablement
object.subscribe(ref)   -> revision notifications
object.serialize(ref)   -> durable form or explicit non-serializable result
```

A view may cache a label and summary, but commands should resolve current state before acting. Revision numbers make stale actions visible. A command can reject “window revision 41 no longer exists” rather than silently act on a new window that reused an XID.

11.3 Presentations are nodes in the widget tree

A presentation is not necessarily a chip. In the basketball prototype, an entire table row, plotted circle, line-chart point, legend label, and watchlist entry can present the same player or game. The presentation metadata should attach to any widget node and determine:

- hover documentation;
- accept highlighting;
- click-to-accept precedence;
- context-command discovery;
- drag/drop semantics;
- inspection and accessibility labels;
- object identity shown in event traces.

```
ui.present(playerRef,
  ui.row(
    ui.text(player.name, {weight: "bold"}),
    ui.metric(player.pts, {label: "PTS"})
  ),
  { primary: "player.focus" }
)
```

The visual child remains ordinary widgets. The wrapper contributes object semantics and default interaction.

11.4 Accept as a scoped interaction protocol

The current single global accept session reproduces the prototype's simplest behavior. A desktop operating environment will need nested and concurrent interactions. A menu command may ask for a file while a background automation runtime is waiting for a window; two seats may interact independently; a modal inspector may constrain acceptance to its subtree.

A richer request is:

```
type AcceptRequest struct {
  ID          SessionID
  Requester   PrincipalID
  Seat        SeatID
  Scope       ScopeRef          // desktop, workspace, surface, subtree
  Types       []string
  Predicate   PredicateSpec
  Cardinality Cardinality         // one, optional, many
  Prompt      string
  Parent      *SessionID
  Deadline    time.Time
}
```

The broker maintains an interaction stack per seat. A child accept temporarily suspends its parent and restores it on completion. A second unrelated requester can either queue, use another seat/scope, or fail with a structured busy result according to policy. The active context is broadcast as state, with a revision, so surfaces highlight compatible presentation nodes without asking the broker for each node.

Accept completion should be explicit about cancellation reasons: user cancelled, requester disconnected, provider disappeared, superseded by parent teardown, timed out, capability denied, or type/predicate mismatch. JavaScript may still map ordinary user cancellation to `null`, while inspection and traces retain the reason.

12. Commands, verbs, and one desktop command graph

The launcher registry, PBUI verbs, key bindings, menu items, and REPL-callable methods currently overlap conceptually. Unify them as command descriptors with typed targets and argument slots. Qtile's command graph demonstrates the value of separating object traversal from invocation and allowing the same abstract call to execute through an in-process or IPC interface. [QTILE-CMD]

12.1 Command descriptor

```
type CommandDescriptor struct {
    ID          string
    Owner       PrincipalID
    Title       string
    Description  string
    Category    []string
    Icon        *ObjectRef
    TargetTypes []string
    Arguments   []ArgumentDescriptor
    KeyHints    []string
    Priority     int
    Undoable    bool
    Effects     []Capability
    Visibility   ConditionSpec
    Enablement  ConditionSpec
}

type ArgumentDescriptor struct {
    Name       string
    Label      string
    Types      []string
    Cardinality Cardinality
    Default    json.RawMessage
    Prompt     string
}
```

A verb is a command with a target object. A launcher entry is a command with no target or a **command** presentation target. A key binding is a gesture mapped to a command and partial arguments. A context menu queries commands applicable to the target and current context. REPL completion searches the same registry. A command palette renders the registry. Documentation is generated from descriptors. Tests can validate argument plans without invoking handlers.

12.2 Invocation protocol

Command invocation should be a small state machine:

```
resolve descriptor
-> authorize principal and effect capabilities
-> resolve target revision
-> fill supplied arguments
-> run accept sessions for missing typed arguments
-> validate final invocation
-> dispatch to command owner
-> receive result, emitted objects, undo record, and events
```

The owner should receive one final invocation object rather than manually nesting arbitrary **pbui.accept** calls for common cases. Handlers may still initiate custom accepts, but declarative argument slots make menus, command-line completion, documentation, and replay coherent.

```
command.define({
  id: "window.move-to-workspace",
  targetTypes: ["window"],
  arguments: [
    {name: "workspace", types: ["workspace"], prompt: "Select destination"}
  ],
  effects: ["wm.mutate"]
}, async ({target, workspace}) => {
  return wm.moveWindow(target.id, workspace.id);
});
```

12.3 Command results are presentations

A command result should be a structured object, not only success/error text:

```
type InvocationResult struct {
    Status      string
    Value       *ObjectRef
    Present     []ObjectRef
    Events      []EventRef
    UndoCommand *Invocation
    Message     string
}
```

The REPL can display the value. The listener can print presentations. A notification can show the message. The trace can link to events. An undo stack can invoke the inverse command when one exists. This aligns command execution with the live-object environment rather than treating commands as fire-and-forget callbacks.

12.4 Command routing and propagation

HyperCard's message path demonstrates one useful idea: an event not handled by the most specific object travels through a defined hierarchy. The widget system should implement a modern, explicit form through capture, target, and bubble phases. Commands can also be resolved through scopes: widget, surface, app runtime, workspace, shell, system. [HYPERCARD]

The propagation path must be visible in traces and controllable. Handlers return **handled**, **continue**, or **prevent-default**; they do not rely on an implicit exception or missing method. A surface can override a global command while explicitly delegating to the parent scope.

13. From row/segment IR to a retained widget tree

The target widget system should preserve the current data-only snapshot boundary while increasing structure and invalidation precision.

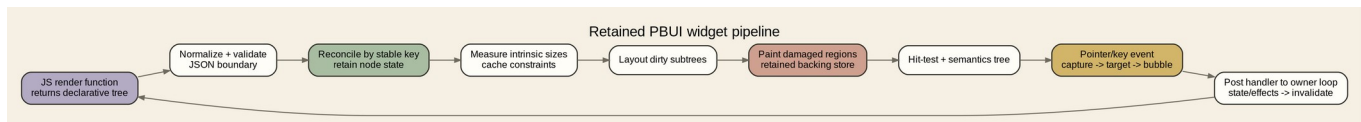


Figure 6. A retained pipeline separates JavaScript composition, normalized identity, measurement, layout, paint, hit testing, and event handling.

13.1 The interchange tree

```
type WidgetNode struct {
    ID          string          `json:"id"`
    Kind        string          `json:"kind"`
    Props       map[string]json.RawMessage `json:"props,omitempty"`
    Children    []WidgetNode             `json:"children,omitempty"`
    Presentation *pbui.ObjectRef          `json:"presentation,omitempty"`
    Handlers    map[string]string        `json:"handlers,omitempty"`
    Semantics   *Semantics               `json:"semantics,omitempty"`
}
```

ID is stable within a surface. **Kind** selects a Go-owned widget implementation. Props are validated against a kind schema. Handler values are opaque IDs registered in the runtime, not function values in the render snapshot. Presentation metadata attaches object semantics. Semantics provides role, accessible label, state, and relationships.

JavaScript builders may produce this tree ergonomically:

```
ui.column({id: "root", gap: 6},
  ui.row({id: "toolbar"},
    ui.button({id: "refresh", label: "Refresh", onPress: refresh}),
    ui.text({id: "status", text: status.value})
  ),
  ui.virtualTable({
    id: "windows",
    rows: windows.value,
    key: row => row.id,
    columns: windowColumns
  })
)
```

The builder registers **refresh** and returns a handler ID. The exported tree remains plain data.

13.2 Reconciliation

The surface host compares the new normalized tree with the committed tree by stable ID and kind:

- Same ID and kind retains widget-local state, measured caches, backing store, scroll position, and focus.
- Prop changes call the widget's update method and return invalidation flags.
- Child order changes update layout and hit-test structure.
- Removed nodes run deterministic teardown, release resources, and invalidate focus or pointer capture.
- Reusing an ID with a different kind is either a replace operation or a validation error according to policy.

This is not a requirement to reproduce React. The host owns reconciliation because it owns layout, paint, and resources. JavaScript only describes the desired tree.

13.3 Measurement and layout

Every widget kind implements a compact protocol:

```
type Widget interface {
    Measure(ctx MeasureContext, constraints Constraints) Size
    Layout(ctx LayoutContext, rect Rect) []Placement
    Paint(ctx PaintContext, damage Region)
    HitTest(point Point) *Hit
    Semantics() Semantics
    Close()
}
```

Measurement results are cached by relevant constraints, theme metrics, font revision, and content revision. A paint-only change does not invalidate measurement. Parent layout is invalidated only when a child's intrinsic size or layout-affecting props change.

Start with deterministic layout primitives rather than a full CSS engine:

- **Row**, **Column**, **Stack**, **Wrap**, **Grid**, **Spacer**, **Rule**, and **Padding**;
- fixed, intrinsic, weighted, min/max, and percentage dimensions;
- alignment, gap, clipping, and overflow;
- **Scroll** and virtualized collections;
- a simple absolute **Canvas** container for charts with Go-owned primitives.

A browser-grade CSS layout engine would add complexity unrelated to the project's core ideas. The protocol should make another layout engine possible later without embedding its semantics into PBUI object identity or runtime APIs.

13.4 Paint and damage

Widgets paint into retained surfaces or layers. Each update returns damage in local coordinates. The surface combines and clips regions, repaints only intersecting nodes, then uploads only damaged rectangles where the backend permits.

```
type Invalidation uint32
const (
    InvalidatePaint Invalidation = 1 << iota
    InvalidateMeasure
    InvalidateLayout
    InvalidateHitTest
    InvalidateSemantics
)
```

Common cases become cheap:

- A blinking cursor invalidates a small rectangle and no layout.
- A focused row changes background and semantics but not measurement.
- A counter with the same digit width invalidates text paint only.
- A new table row invalidates virtual-list extent and visible row layout, not every offscreen row.
- A theme color change invalidates paint; a theme font-size change invalidates measure and layout.

13.5 Hit testing and event revisions

The hit-test tree is derived from a committed layout revision. A pointer event carries that revision and a path of stable node IDs. The event posts to the runtime owner. Before invoking a handler, the runtime checks whether the node and handler still exist. When a new render commits before the event runs, policy can retarget by stable ID or reject the stale event with a trace. This prevents a click from invoking the handler that happens to occupy the same row index in a new tree.

The event path supports:

- capture handlers from surface root to target;
- target handler;
- bubble handlers from target to root;
- default action such as presentation accept, button activation, focus, selection, or menu opening;

- pointer capture for drags;
- keyboard focus scopes and traversal;
- text input separate from physical key events;
- cancellation when a modal or interaction scope changes.

13.6 Initial widget inventory

The initial inventory should be selected by the shell and developer-workbench requirements, not by generic GUI completeness.

Group	Widgets	Immediate use in go-go-wm
Text and structure	Text, RichText, Row, Column, Stack, Wrap, Grid, Padding, Spacer, Rule, Panel	Bars, menus, forms, inspectors, title content.
Input	Button, Toggle, RadioGroup, TextField, TextArea, Select, Slider, Splitter	Launcher, settings, REPL editor, resize controls.
Collections	List, VirtualList, Table, VirtualTable, Tree, Tabs, Breadcrumbs	Window lists, runtime lists, object paths, command results, traces.
Data display	Metric, Progress, Sparkline, LinePlot, ScatterPlot, BarPlot, Heatmap, Timeline	Resize profiler, event rates, CPU/memory, layout visualizer.
Object tools	Presentation, ObjectChip, Inspector, PropertyGrid, WatchList, DiffView	PBUI core, REPL outputs, windows, workspaces, runtime state.
Shell	Menu, MenuItem, CommandPalette, PopupAnchor, Modal, Tooltip, Notification, StatusItem	Right-click menus, taskbar, top bar, command launcher.
Advanced	Canvas, Image, Markdown, CodeEditor, TerminalEmbed	Domain apps and developer environment after the core stabilizes.

Charts should begin as high-level data specifications rendered in Go. Arbitrary JavaScript pixel loops would reintroduce runtime calls and make damage, accessibility, and performance difficult. A controlled canvas can later expose retained drawing objects rather than an immediate-mode callback.

14. One surface runtime for tiles, bars, menus, modals, and taskbars

A surface combines a widget tree with platform policy. The same widget tree can be shown as a tiled application, standalone window, popup, bar, modal, notification, or inspector. The role changes placement, stacking, focus, lifetime, and protocol properties, not the widget API.

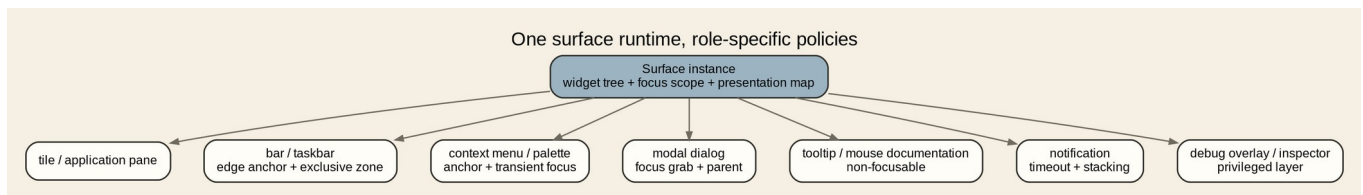


Figure 7. Surface roles share rendering and interaction while policy remains role-specific.

14.1 Surface descriptor

```
const topbar = ui.surface({
  id: "shell.topbar",
  role: "bar",
  output: "primary",
  anchor: {edge: "top"},
  exclusive: 28,
  layer: "top",
  focus: "none",
  capabilities: ["ui.global"],
  render() { return shellBar(); }
});
```

A complete descriptor includes:

```

type SurfaceSpec struct {
    ID          SurfaceID
    Owner        PrincipalID
    Role         SurfaceRole
    Output       OutputSelector
    Anchor       AnchorSpec
    InitialSize  SizeSpec
    ExclusiveZone int
    Layer        Layer
    FocusPolicy  FocusPolicy
    InputRegion  RegionSpec
    Parent       *SurfaceID
    Lifetime     LifetimePolicy
    Modal        bool
    PersistKey   string
    ThemeScope   string
}

```

The shell validates that the owner has the capability for the requested role and layer. An ordinary app can create a tile, window, or child popup. Only a trusted shell runtime can reserve screen edges, create global overlays, intercept system key scopes, or place surfaces above security-sensitive modals.

14.2 Placement and anchoring

Popup and menu placement should use an anchor object rather than raw screen coordinates:

```

ui.popup({
  id: "window-menu",
  anchor: {presentation: windowRef, edge: "bottom-start"},
  constrainTo: "workarea",
  flip: true,
  content: commandMenu(windowRef)
});

```

The host resolves the presentation node's current rectangle at the committed layout revision. It flips or shifts the popup within the work area. When the anchor disappears, the popup closes. This supports right-click menus, completion lists, tooltips, and object inspectors consistently.

14.3 Focus scopes and modal stacks

Each surface can define a focus scope. A menu traps navigation within its items and restores the prior focus on close. A modal surface blocks default interaction with its parent scope and may own an accept child session. A tooltip owns no focus. A bar may allow mouse interaction but not take keyboard focus. A REPL notebook has an internal text focus independent of the selected desktop window.

The modal stack belongs to the shell, not to scripts. Scripts request a modal surface; the shell enforces ordering, parent relationships, focus restoration, cancellation, and teardown when the owner runtime dies.

14.4 Bars and taskbars as object browsers

A presentation-oriented taskbar should not be a fixed list of window titles. It is a filtered object browser over windows, workspaces, commands, runtimes, notifications, and user-defined status objects. Each entry is a presentation. Right-click commands are type-directed. Accepting a `window` can be answered from the taskbar. A script can replace or augment the view without replacing the underlying object and command protocols.

A default bar might compose:

```

ui.row({id: "bar",
  WorkspaceStrip({source: wm.workspaces}),
  WindowTrail({source: wm.focusHistory}),
  ui.spacer({grow: 1}),
  RuntimeHealth({source: system.runtimes}),
  NotificationSummary({source: system.notifications}),
  Clock({format: "HH:mm"})
})

```

The shell owns the exclusive screen zone and global lifecycle. The trusted runtime owns composition. Individual status widgets may be contributed by lower-privilege runtimes through declared slots and bounded snapshots.

14.5 Context menus are command views

A context menu should not be assembled independently in every widget. It queries the command graph with target, current input context, principal, surface, workspace, and seat. The result includes grouping, ordering, enablement, shortcuts, argument status, and provenance. The menu renders that data and invokes the command protocol.

This design provides several benefits:

- a command appears consistently in every presentation of the object;
- disabled commands can explain why they are disabled;
- menus and palettes remain inspectable and scriptable;
- command conflicts and ownership are visible;
- the REPL can query exactly which commands a right-click would show;
- tests operate on command data without simulating pixels.

14.6 Shell composition and failure containment

The shell should have a minimal native fallback surface. A syntax error or runtime crash in the trusted shell script must not leave the user without a launcher, exit command, or runtime inspector. Native fallback capabilities should include:

- open emergency launcher or REPL;
- list and stop runtimes;
- switch workspaces and focus windows;
- reload or roll back shell configuration;
- close or restart the WM cleanly;
- show script errors and capability denials.

The normal bar, menus, and taskbar can be scripted. The recovery path must be native and small.

Part V. JavaScript runtimes and the REPL as OS building blocks

15. Treat JavaScript runtimes as desktop processes

The phrase “fully scriptable” can mean two incompatible designs. In the first, one privileged configuration runtime has direct access to most shell mechanisms. It is easy to build and easy to freeze or corrupt. In the second, scripts run as supervised desktop processes with explicit identities, capabilities, queues, state, and lifecycle. The second design is required once scripts host persistent widgets, menus, taskbars, automation, or third-party packages.

The operating system below go-go-wm remains Linux and X11. The project is building a **user-space desktop operating environment**: an object, command, UI, and runtime layer in which users can create and inspect tools while the system runs. Precise terminology matters because kernel isolation, filesystem permissions, networking, devices, and process accounting remain external responsibilities. go-go-wm can still impose meaningful application-level capability and failure boundaries inside its own environment.

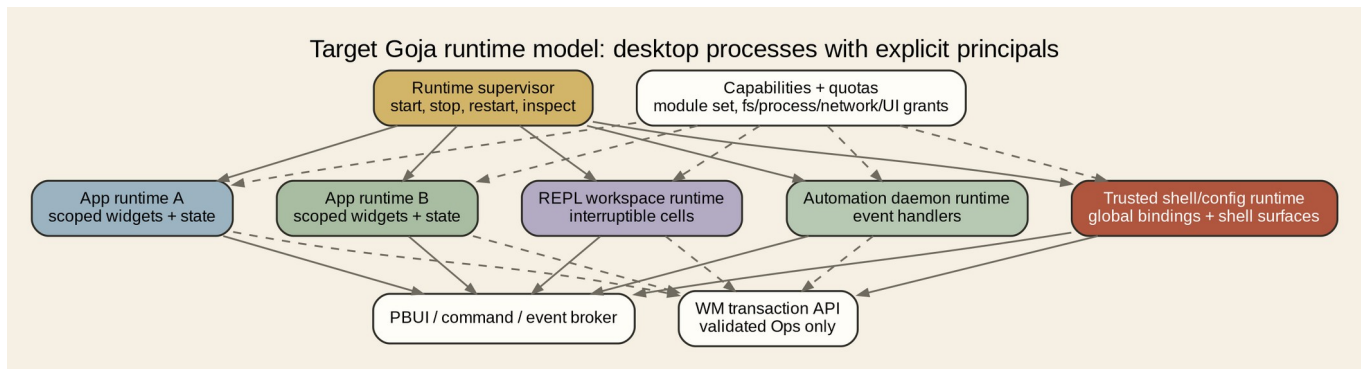


Figure 8. Multiple supervised runtimes replace one global scripting world.

15.1 Runtime classes

Use several runtime classes with different trust and persistence policies.

Runtime class	Typical lifetime	Default authority	State model	Failure policy
Shell/config runtime	WM session	Global key bindings, shell surfaces, command registration, controlled process launch	Versioned shell configuration and state	Restart with native fallback; allow rollback to last valid revision.
Application runtime	App/package lifetime	Own surfaces, scoped PBUI objects/commands, selected data providers	App-defined serializable state with migrations	Restart independently; close or restore owned surfaces.
Automation runtime	Daemon lifetime	Event subscriptions and explicitly granted commands	Small durable state/checkpoints	Backoff and restart; disable after repeated failure.
REPL workspace runtime	Notebook/workspace lifetime	Read-mostly inspection, user-granted effects	Notebook source plus serializable bindings/outputs	Interrupt cell; restart and replay when VM health is uncertain.
One-shot macro runtime	One invocation	Narrow command-specific capabilities	None or invocation-local	Terminate after result or deadline.
Preview/test runtime	Hot-reload revision	No global effects by default	Temporary	Destroy on validation failure; swap in only after successful render/tests.

A runtime may be implemented in-process with its own Goja VM and owner loop. “Process” here is a semantic boundary rather than necessarily a Linux process. The descriptor and supervisor should make it possible to move selected runtimes out of process later without changing command, object, or surface protocols.

15.2 Runtime descriptor and principal

```
type RuntimeSpec struct {
    ID             RuntimeID
    Principal       PrincipalID
    Package         PackageID
    Class           RuntimeClass
    Entry           SourceRef
    Modules         []string
    Capabilities     CapabilitySet
    Limits           RuntimeLimits
    PersistenceKey   string
    Restart         RestartPolicy
}

type RuntimeLimits struct {
    MaxEvalTime      time.Duration
    MaxHandlerTime    time.Duration
    MaxQueueDepth     int
    MaxOutstandingIO int
    MaxTimers         int
    MaxSurfaces       int
    MaxWidgetNodes    int
    MaxSurfacePixels   int64
    MaxRenderHz       float64
    MemorySoftBytes    int64
}
```

The principal is generated by the supervisor and remains the authority identity for broker registrations, commands, object providers, and surfaces. Package name and human-readable runtime name are metadata. A restarted runtime may receive a new runtime instance ID while retaining a stable package principal or capability grant identity according to policy.

15.3 The runtime supervisor

The supervisor owns the full lifecycle:

```
created -> loading -> starting -> running
                                     |
                                     | -> stopping -> stopped
                                     | -> failed -> backoff -> restarting
                                     | -> quarantined
                                     +-> rejected
```

It records:

- source revision and package version;
- granted capabilities and who granted them;
- module instances and provider cleanup;
- owner-loop health and last heartbeat;
- queue depth, handler latency, timer count, outstanding promises, and render rate;
- owned commands, object providers, subscriptions, and surfaces;
- persisted state version and migration history;
- recent errors, interrupts, restarts, and dropped/coalesced events.

Every field should be inspectable through a `runtime` presentation. The runtime's menu can offer Stop, Restart, Reload, Inspect Queues, Inspect Capabilities, Open Logs, Open Source, Save State, and Revoke Grant according to caller authority.

15.4 Capabilities instead of one `allow-exec` switch

The current `--allow-exec` boundary is a useful prototype safeguard but too broad for packages. Define granular capabilities with optional resource constraints:

```

wm.read
wm.mutate.layout
wm.mutate.focus
wm.mutate.workspace
wm.mutate.window-state
process.spawn(command-pattern or package launcher only)
fs.read(path grants)
fs.write(path grants)
network.connect(host/port grants)
clipboard.read
clipboard.write
notifications.publish
pbui.type.register
pbui.command.register
pbui.object.provide
pbui.accept.request
ui.surface.tile
ui.surface.window
ui.surface.popup
ui.surface.global-bar
ui.surface.overlay
ui.input.global-binding
runtime.inspect.self
runtime.inspect.others
runtime.manage

```

Capabilities should be checked at the effect boundary, not only when a module is loaded. A runtime may have the `wm` module for read queries without permission to mutate focus. A command descriptor declares expected effects, allowing the launcher or menu to show a permission prompt before execution rather than failing halfway through.

A grant includes scope and provenance:

```

type CapabilityGrant struct {
    Capability string
    Scope      json.RawMessage
    GrantedBy  PrincipalID
    Reason     string
    Expires    *time.Time
    Revision   uint64
}

```

The capability UI itself should be built from presentations and commands. A package request is a typed `capability-request` object. The user can inspect the package, requested scope, and command effects before granting. Grants are persistent data, not hidden flags in a startup command.

15.5 Effects and transactions

JavaScript handlers should not receive mutable Go objects. They issue effects through modules. Effects are validated, authorized, queued to the owning subsystem, and return structured results.

```

async function moveFocusedWindow() {
    const target = await pbui.accept("workspace", "Move focused window to...");
    if (!target) return;
    return wm.transaction(tx => {
        tx.moveWindow(wm.focusedWindow(), target);
        tx.focusWorkspace(target);
    });
}

```

`wm.transaction` should construct data on the JS owner loop and submit one batch. It should not hold locks or call back into JavaScript while the WM applies it. The result includes operation results, events, revision, and partial failure information. Transactions that need atomicity should validate against one model revision before any effects are applied.

For cross-subsystem actions, use a saga-like command result rather than pretending X11, filesystem, process spawn, and UI changes can be ACID. Each effect records completion and compensation where possible. The trace should show partial outcomes.

15.6 Scheduling priorities

A live environment has several work classes competing for owner loops and the WM loop. Define priorities rather than relying on channel arrival order.

1. X input and cancellation
2. Geometry and focus commits
3. Expose recovery and visible paint within budget
4. WM command/control replies
5. Surface event handlers for visible focused surfaces
6. Surface renders and background data refresh
7. Automation handlers
8. Telemetry, indexing, and maintenance

A runtime owner loop can remain single-threaded while its scheduler selects queued tasks by class and deadline. Long host operations must leave the owner loop. JavaScript itself remains single-threaded per runtime, preserving Goja's requirements.

15.7 Timeouts and interruption

Context cancellation does not automatically stop synchronous JavaScript already executing. Every evaluation or handler that can run arbitrary code needs an interrupt path.

The supervisor should:

37. start a task with a deadline and task ID;
38. request cooperative cancellation through task context and promise APIs;
39. after a short grace period, call `goja.Runtime.Interrupt` with a typed interruption value;
40. catch the interruption at the owner boundary and mark the task cancelled or timed out;
41. run a health probe before accepting more work;
42. restart the runtime when invariants or host modules may have been left in an uncertain state.

Host functions must also honor context. A Go host function that blocks in a socket call cannot be interrupted by `VM.Interrupt` until it returns. All potentially blocking host work should run outside the owner loop and return promises.

15.8 Memory and runaway allocation

Goja does not provide a simple per-runtime hard heap limit inside one Go process. Use layered controls:

- cap exported JSON size and widget node count;
- cap surface dimensions and total retained pixels;
- cap timers, subscriptions, pending promises, and queued tasks;
- record allocation deltas around tasks as a soft signal;
- restart runtimes that exceed soft limits repeatedly;
- use separate Linux processes for untrusted or memory-intensive packages when hard containment is required;
- make object providers return references and paged data rather than large copies.

The protocol must not imply security guarantees stronger than the implementation. In-process capabilities prevent accidental or policy-violating host calls through supported APIs; they do not make arbitrary native Go extensions safe against memory corruption or reflection escapes.

16. State, persistence, hot reload, and packages

A live environment needs persistence, but persisting the entire Goja heap is neither necessary nor robust. Persist source, declarative package metadata, serializable application state, notebook inputs, durable object references, command/layout data, and selected outputs. Reconstruct runtimes through controlled replay.

16.1 State stores

Each runtime gets a versioned key/value state store with transactional updates:

```
const state = system.state.open("my-package", {
  version: 3,
  defaults: {count: 0, projects: []},
  migrate: {
    1: old => ({...old, projects: []}),
    2: old => ({...old, count: Number(old.count) || 0})
  }
});
```

The host validates that stored values are serializable and enforces size quotas. State changes emit revisions. Widgets can subscribe to selectors. The store is not a transparent proxy to arbitrary runtime objects; closures, DOM-like nodes, Goja values, open files, and X handles cannot be persisted directly.

16.2 Hot reload protocol

Hot reload should be a transaction:

43. Load new source in a preview runtime with the proposed capabilities.
44. Validate module imports, command descriptors, type declarations, and surface trees.
45. Run package self-tests or smoke hooks with effects disabled or sandboxed.
46. Ask the old runtime for a versioned serializable handoff state.
47. Start the new runtime with migrated state.
48. Reconcile exported commands, object providers, and surfaces by stable package IDs.
49. Atomically switch ownership in the registries.
50. Stop the old runtime after the new revision is healthy.
51. Roll back when startup, migration, or initial rendering fails.

Stable surface and widget IDs allow focus, geometry, scroll position, and retained resources to survive reload. Handler identities change with the runtime revision, so in-flight events must target the revision that produced their hit path or be rejected.

16.3 Package manifest

```
id: dev.go-go-golems.profiler
version: 0.4.0
entry: main.js
runtime: application
modules: [pbui, ui, wm, system]
capabilities:
  - wm.read
  - ui.surface.tile
  - pbui.command.register
state_version: 2
exports:
  commands: [profiler.open, profiler.capture]
  types: [profile, resize-trace]
  surfaces: [profiler.main]
```

The package manager records source, signature or origin, dependencies, requested capabilities, grants, state schemas, exported object types, and command IDs. Packages should be inspectable objects. Installation, update, enable, disable, grant, and rollback are commands with visible results.

16.4 Shared services versus direct package imports

Packages should share data through typed object providers, commands, and events rather than importing one another's runtime-local state. Direct JavaScript package imports remain useful for pure libraries. Stateful integration belongs at protocol boundaries:

- a Git package provides `git.repository` and `git.commit` objects;
- a project dashboard accepts those objects and queries documented provider methods;
- a command package registers checkout, diff, and open commands for the types;
- a REPL inspector renders any compatible object;
- a taskbar can show repository status through a contributed status object.

This preserves the PBUI composability demonstrated by the prototypes while making provider lifetime and capabilities explicit.

17. The REPL as the operating-environment control plane

The rich REPL should be treated as a permanent system tool, not an optional demonstration. It is the one surface where source, values, commands, objects, traces, and runtime state naturally meet. In a presentation-oriented environment, its output cells are not a transcript of strings. They are durable interaction nodes that can be inspected, accepted, linked, watched, and reused.

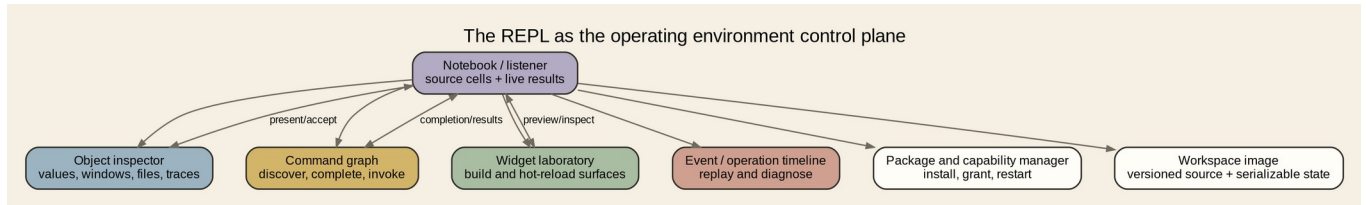


Figure 9. The REPL connects object inspection, command discovery, widget development, event timelines, package management, and persistent workspace state.

17.1 Cell model

```
type Cell struct {
  ID          CellID
  Number      int
  RuntimeRev  uint64
  Source      string
  SourceMap   *SourceMap
  State       CellState
  StartedAt   time.Time
  EndedAt     time.Time
  Console     []ConsoleEntry
  Result      *ObjectRef
  Views       []ViewSpec
  Error       *ObjectRef
  Effects     []EffectRef
  Events      []EventRef
  Dependencies []CellID
}
```

Cell state and execution metadata should be visible. A running cell shows elapsed time, cancellation control, current promise count, and perhaps the host operation it is awaiting. A completed cell exposes source, result, derived views, console, effects, and events as presentations. A failed cell exposes a structured error and stack frames. A cancelled cell records whether cancellation was cooperative, interrupted, or required runtime restart.

17.2 Two execution modes

A stateful REPL is useful because bindings and live objects persist. Reproducibility is useful because restart and sharing require replay. Support two explicit modes:

- **Live mode** evaluates into one long-lived runtime. It offers immediate incremental work and may contain non-serializable values.
- **Replayable notebook mode** records dependencies, constrains side effects through declared transactions, and can rebuild the runtime by replaying cells from a checkpoint.

A notebook can mix modes by marking cells as transient or effectful. The UI should make the distinction visible rather than pretending all state can be reproduced.

17.3 Interruption and restart

Add Stop to the keyboard and cell chrome. The cancellation sequence is:

52. mark the cell **cancelling** and cancel host-operation contexts;
53. wait a short cooperative interval;
54. interrupt synchronous Goja execution;
55. mark pending promises rejected with a cancellation object;
56. run a health check that evaluates a minimal expression and checks module registries;
57. when unhealthy or when host code may have partially mutated runtime-local registries, restart the workspace runtime;
58. restore state from the most recent checkpoint and replay eligible cells;
59. mark non-replayable cells and ephemeral references as stale.

The user must never wonder whether Stop worked. State transitions and restart progress belong in the cell and runtime inspector.

17.4 Universal object inspector

The inspector is a protocol-driven set of views, not `JSON.stringify` with styling. An object provider can expose sections:


```
type InspectorSection struct {
    ID      string
    Title   string
    Kind    string // fields, list, table, tree, code, image, plot, timeline
    Data    json.RawMessage
    Commands []string
    Lazy    bool
}
```

Default sections include identity, summary, properties, commands, relationships, history, source/provider, serialization, and raw snapshot. Specialized providers add domain sections. A window shows geometry, X properties, workspace, focus/stack history, protocol messages, and applicable commands. A widget shows props, measured size, layout rectangle, damage, handlers, presentation object, and render time. A runtime shows queues, tasks, capabilities, owned resources, and errors.

Inspector navigation itself produces presentation objects. A property value can be dragged or accepted into another command. A stack frame links to source. A window rectangle links to a visual debug overlay. A trace event links to the command and objects that caused it.

17.5 Completion and discovery

Completion should draw from structured metadata:

- module exports and TypeScript declarations generated by xgoja providers;
- runtime bindings and object properties;
- command graph paths and argument descriptors;
- PBUI type registry;
- package registry;
- recent object and command history;
- valid widget kinds, props, enums, and event names.

The completion popup is a PBUI surface. Each completion candidate is a typed object such as `js-symbol`, `command`, `ptype`, or `package`. Right-click or a documentation key opens its inspector. Accepting a command or object can insert a source expression rather than only invoke it.

17.6 Accept into source

The REPL should make the presentation system visible in source editing. A user can write:

```
const w = await pbui.accept("window");
```

and click a window. A faster interaction is an explicit typed hole:

```
wm.focus(⟨window⟩)
```

Activating the hole starts an accept. The selected object inserts a stable source representation such as `pbui.ref("window", "wm:42", 7)` or a readable helper `wm.window("terminal-3")` when one exists. The source remains inspectable and replayable. The UI should not silently insert a large JSON snapshot.

17.7 Results as live views

The existing derivation system is a good start. Extend it with provider-driven view selection and user preferences:

- numbers: scalar, gauge, history, unit-aware formatting;
- sequences: table, list, sparkline, histogram, raw values;
- datasets: virtual table, schema, summary statistics, plots;
- colors/palettes: swatches and contrast checks;
- windows/workspaces: miniature layout diagrams and command panels;
- runtime traces: timelines, queue charts, flamegraph links;
- widget trees: hierarchy, rectangles, damage overlay, render costs;
- errors: stack frames, source, locals, related events, retry controls.

A cell can cycle views without reevaluating. View state belongs to the notebook presentation, not the raw value.

17.8 Debugger and event timeline

The first debugger can be host-oriented rather than a full source-level JavaScript debugger. It should expose:

- currently running task and owner-loop queue;
- recent task durations and interruptions;
- promise/host-operation states;

- event deliveries and handler outcomes;
- emitted commands, operations, and effects;
- script errors with stack and source map;
- runtime restarts and state replay.

The event timeline is canonical infrastructure. Every event has sequence, source principal, cause, related command, related objects, logical revision, and timestamp. The REPL can filter and watch streams:

```
watch(pbui.events("window.*"), {view: "timeline"})
watch(system.metrics("resize"), {view: "latency-chart"})
watch(wm.windows(), {key: w => w.id, view: "table"})
```

Watch values are presentations with Pause, Resume, Snapshot, Export, Open Producer, and Create Automation commands.

17.9 REPL and shell development workflow

A productive workflow is:

60. Explore objects and command APIs in a REPL workspace.
61. Build a widget tree in a preview surface.
62. Inspect layout, damage, events, and performance live.
63. Convert working code into a package or shell module.
64. Declare capabilities and state migrations.
65. Hot-load the package in a preview runtime.
66. Run smoke checks and swap it into the shell.
67. Retain the notebook as executable documentation and regression evidence.

This is the strongest connection to live Smalltalk environments: the development tools operate on the running object system and can be extended from within it. The modern requirement is that every live mutation also has provenance, a capability check, and a recoverable revision.

18. The operating-environment service map

The long-term environment can be described as ten services. They may begin in one binary and one Unix socket, but their contracts should be explicit.

Service	Owns	Primary API objects
Window service	X11 clients, geometry, workspaces, focus, stacking, fullscreen, outputs	window, workspace, output, layout-node
Presentation service	Type registry, object references, providers, view lookup	ptype, object-ref, provider
Command service	Descriptors, invocation, argument acceptance, enablement, history, undo	command, invocation, command-result
Interaction service	Seats, focus scopes, accept stacks, pointer captures, modal scopes	seat, accept-session, focus-scope
Surface service	Surface roles, widget trees, layout, damage, hit testing, semantics	surface, widget, damage-region
Runtime service	Goja owners, capabilities, queues, lifecycle, metrics, restart	runtime, task, capability-grant
Event service	Canonical events, state streams, telemetry, subscriptions	event, stream, subscription
State service	Versioned stores, checkpoints, migrations, notebook persistence	state-store, checkpoint, notebook
Package service	Source, dependencies, manifests, grants, install/update/rollback	package, package-revision
Inspection service	Universal object views, traces, source links, debug overlays	inspector, trace, source-location

The REPL is a client of all ten services and exposes them through one coherent notebook surface. The shell runtime composes surfaces and commands from the same services. External programs can participate through the PBUI/command protocols without embedding Goja.

Part VI. Engineering roadmap, tests, and intern curriculum

19. Observability and correctness architecture

A live, scriptable shell has more failure modes than a conventional tiling manager. Observability must therefore be part of the object model rather than a collection of log files.

19.1 Canonical event envelope

```
type Event struct {
    ID          EventID
    Seq         uint64
    Time        time.Time
    Source       PrincipalID
    Cause        *EventID
    Command      *InvocationID
    ModelRev     uint64
    Kind         string
    Objects      []pbui.ObjectRef
    Data         json.RawMessage
    Reliability  EventReliability
}
```

Every operation event links to the command or input that caused it. Runtime errors link to the task and source location. Paint traces link to the surface and widget. X errors link to the requested platform effect and model revision. These relationships allow the inspector to move from a visible problem to the responsible state transition.

19.2 Structured debug overlays

The manager should provide native overlays toggled by commands:

- frame, client, title, border, divider, and popup rectangles;
- node IDs, workspace IDs, XIDs, and model revisions;
- focus target, surface focus path, modal scope, and pointer capture;
- dirty geometry roots and damaged paint regions;
- widget bounds, stable IDs, layout constraints, and hit path;
- X request counts and current/applied state differences;
- script runtime owner, handler ID, event queue depth, and render revision.

The overlay itself should avoid the normal script renderer so it remains useful when the widget system fails. Its data should also be exposed as presentations for the REPL.

19.3 Testing layers

Pure unit and property tests

Use pure tests for tree operations, layout arithmetic, directional neighbor selection, dirty-set derivation, focus/fullscreen/modal decisions, command argument planning, type compatibility, widget measurement/layout, reconciliation, event propagation, capability checks, state migrations, and runtime lifecycle decisions.

Property tests should cover:

- applying a valid operation preserves tree invariants;
- serialize/deserialize round-trips preserve identity and layout;
- layout rectangles partition the parent according to divider rules without negative sizes;
- operation replay reaches the same model and revision-independent state;
- command argument filling is deterministic;
- widget reconciliation preserves state only for matching stable identity;
- damage never escapes a surface and final full repaint equals incremental repaint;
- provider reference encode/decode is bijective for valid data;
- state migrations are monotonic and idempotent where required.

Model-based state-machine tests

Generate sequences of manage, focus, split, float, fullscreen, switch, destroy, popup, modal, runtime crash, and accept actions. Compare the implementation state against a small reference model. The GGWM-010/011 focus bugs are exactly the class model-based testing catches because they arise from unusual action ordering.

Xvfb protocol fixtures

Create small synthetic clients with controlled hints and behavior:

- client supporting or rejecting `WM_DELETE_WINDOW`;
- fixed-size, resize-increment, aspect-constrained, transient, utility, splash, dock, and override-redirect windows;
- client that issues configure requests repeatedly;
- client that is slow to redraw;
- client that changes title/class/type after mapping;
- client that destroys or reparents itself during a drag;
- client that requests fullscreen and active-window state through EWMH;
- two monitors through XRandR in Xephyr where supported.

Assert properties and geometry over the control API and X queries. Use screenshot pixel assertions only for rendering behavior, not as the sole state oracle.

Golden rendering tests

The project already has golden images for bars, titles, launchers, and plots. Extend them to widget kinds and theme combinations. Add a second class of incremental-render tests: render a full surface, apply a small change through damage, and compare the resulting pixels to a fresh full render.

Script/runtime tests

Test each module through the same provider path used in production. Required cases include:

- owner-loop enforcement and callback posting;
- cancellation before start, during host I/O, and during synchronous JS;
- runtime restart and cleanup of verbs, commands, providers, surfaces, and subscriptions;
- duplicate human-readable runtime names;
- capability grant/deny and revocation during execution;
- queue overflow policy by event class;
- invalid widget trees preserving the previous committed frame;
- hot reload success, migration failure, and rollback;
- state size and widget-node quotas;
- REPL replay with serializable and stale ephemeral outputs.

Long-running soak tests

Run hours-long scripted sessions that repeatedly create and destroy workspaces, clients, script surfaces, menus, accepts, and runtimes. Track X resource counts, shared memory segments, goroutines, heap, broker clients, registered commands, widget nodes, and event queue depths. Resource counts should return to a stable baseline after each cycle.

19.4 Failure injection

Add debug hooks to inject:

- X errors on selected resource operations;
- SHM allocation and attach failure;
- broker disconnects and delayed replies;
- runtime task timeout and forced interrupt;
- provider disappearance during object resolution;
- surface render failure after a valid prior revision;
- state migration error;
- command owner crash during invocation;
- event subscriber backlog;
- client destruction during geometry transaction.

Each injected failure should produce a structured error object, clean resource ownership, and leave the native recovery shell usable.

20. Phased roadmap

The phases below are designed so each one produces a useful system and a stronger foundation for the next.

Phase 0: responsiveness and control-plane safety

Goal: eliminate the clearest interactive stalls and make failures interruptible and inspectable.

Work:

- Add resize tracing, latency distributions, X request counters, paint pixels, upload bytes, and coalescing counts.
- Build `LayoutSnapshot` and current/applied geometry caches.
- Replace divider throttling with a latest-only scheduler and one committed ratio operation per drag.
- Add a geometry-only resize path and defer builtin/script repaint during drag.
- Skip unchanged divider/bar paint and duplicate map/configure work.
- Handle tiled client `ConfigureRequest` through cached geometry and synthetic notification rather than full relayout.
- Add per-cell REPL cancellation and Goja interruption, non-blocking evaluation enqueue, rune-aware editing, and visible runtime health.
- Give broker connections unique principals; split reliable control from best-effort event delivery.
- Add native runtime and error inspector fallback commands.

Exit criteria:

- Terminal-only resize meets the initial latency budget with near-zero manager paint bytes.
- An infinite REPL loop is interruptible without restarting the WM.
- A slow broker subscriber cannot lose accept results or freeze unrelated clients.
- Current implementation behavior remains covered by regression tests.

Phase 1: resource separation and retained surfaces

Goal: establish the platform and rendering model required for a rich scripted shell.

Work:

- Split title/border resources from client frame interiors.
- Introduce retained `Surface` objects with damage and render cost history.
- Implement widget protocol v1 with stable IDs, Row/Column/Stack/Grid/Text/Button/Presentation/List/Table/Scroll/Plot primitives.
- Separate measure/layout/paint/hit-test/semantics invalidation.
- Add surface roles for tile, window, popup, menu, modal, tooltip, notification, and bar.
- Add focus scopes, modal stack, popup anchoring, pointer capture, and capture/target/bubble event routing.
- Add runtime supervisor, runtime identities, capability checks, quotas, and cleanup registries.
- Convert blocking JavaScript host APIs to asynchronous promises.
- Add object handles/providers and broker-scoped accept stacks.

Exit criteria:

- A JS-defined top bar, context menu, modal, taskbar, and dashboard use one widget/surface runtime.
- A counter update damages a small region; a resize does not force full widget repaint.
- Killing an app runtime closes its surfaces, removes commands/providers, and leaves the shell responsive.
- A script package cannot spawn processes or create a global bar without grants.

Phase 2: command graph and live development environment

Goal: make the environment coherent and discoverable from the REPL.

Work:

- Unify launcher entries, verbs, menu items, and script commands under command descriptors and typed argument slots.
- Generate menus, palettes, completion, help, and key-binding inspection from the command graph.
- Build the universal inspector, source-linked errors, runtime/task views, widget-tree inspector, and event timeline.
- Add REPL checkpoints, restart/replay, completion metadata, typed source holes, watches, and persistent notebooks.
- Add package manifests, state stores, migrations, preview runtimes, hot reload, rollback, and capability UI.
- Make bars and taskbars object browsers over windows, workspaces, commands, runtimes, and status providers.
- Complete baseline ICCCM/EWMH support and add multi-output work-area policy.

Exit criteria:

- The same command can be discovered and invoked from a context menu, launcher, key map, command palette, and REPL.
- A developer can inspect a slow widget from its visible node through render traces to the owning runtime and source.
- Shell code can be edited in a notebook, validated in preview, hot-swapped, and rolled back.
- Notebooks and package state survive restart without serializing raw Goja heap state.

Phase 3: mature desktop environment

Goal: support sustained daily use and third-party extension without weakening the core.

Work:

- Add robust multi-monitor/output topology, per-output workspaces or documented global workspace policy, hotplug, scale, and work areas.
- Add accessibility semantics, keyboard-only traversal, text services, clipboard protocols, and IME integration.
- Add virtualized large data widgets, richer plotting, code editor integration, and controlled canvas layers.
- Add out-of-process runtime option for hard isolation and memory limits.
- Add package signing/origin policy, dependency locking, reproducible builds, and import/export of workspace images.
- Add session restore with client matching and explicit stale-resource handling.
- Evaluate a Wayland backend only after model, command, runtime, and surface contracts are platform-independent.

Exit criteria:

- Daily shell configuration and common tools can be written as packages without native code.
- Third-party packages have inspectable permissions and independent failure boundaries.
- The same logical model and script APIs can drive another display backend without embedding X11 types.

21. Intern curriculum and evidence-driven labs

The project is unusually suitable for an educational development process because every concept can be connected to a trace, object, or visual result. Each lab below requires evidence, not only code completion.

Lab 1: follow one client from MapRequest to focus

Purpose: understand X11 ownership, reparenting, hints, and state maps.

Tasks:

68. Add a trace ID to one synthetic test client.
69. Record MapRequest, property reads, classification, frame creation, reparent, configure, map, focus, and EWMH publication.
70. Render the trace as a timeline and link each event to the client/frame objects.
71. Destroy the client and prove that every callback, buffer, pixmap reference, map entry, and object provider is released.

Evidence:

- a sequence diagram generated from the trace;
- before/after X resource and map counts;
- a test for idempotent teardown;
- an explanation of which window receives each event and why.

Lab 2: prove layout invariants with generated operations

Purpose: learn the pure model before X11.

Tasks:

72. Generate random valid split, close, swap, move, and workspace operations.
73. Assert unique node IDs, valid ratios, no missing children, correct leaf counts, and deterministic serialization.
74. Compute layout at several screen sizes and assert rectangle partition rules.
75. Minimize a failing sequence and present it as a `wm-op-sequence` object.

Evidence:

- property-test output;
- a visual tree and rectangle view for the minimized sequence;
- the exact invariant that failed.

Lab 3: instrument and remove one resize cost

Purpose: distinguish layout, X requests, painting, upload, and client work.

Tasks:

76. Capture a baseline scripted divider drag.
77. Add one optimization, such as eliminating recursive frame lookups or duplicate map requests.
78. Compare p50/p95/p99 latency, samples, X requests, paint pixels, allocations, and final pixels.
79. Explain why CPU reduction did or did not improve visible latency.

Evidence:

- before/after trace tables and plots;
- pprof or runtime-trace evidence;
- a regression test preventing the eliminated work.

Lab 4: add a presentation type and command

Purpose: understand PBUI composition across tools.

Tasks:

80. Define a `source-location` object provider.
81. Present source locations in REPL errors, trace events, and an inspector.
82. Register Open, Copy, Inspect File, and Set Breakpoint commands.
83. Define a command that accepts another `source-location` and compares them.

Evidence:

- the same object accepted from at least three surfaces;
- a menu generated from the command registry;
- disconnect cleanup and stale-provider behavior.

Lab 5: build a script-defined status bar

Purpose: exercise surface roles, retained widgets, and runtime isolation.

Tasks:

84. Create a trusted shell runtime bar with workspace presentations, focused window, runtime health, and clock.
85. Update the clock without relaying out workspace chips.
86. Crash the clock widget's contributing runtime and preserve the bar.
87. Inspect damage rectangles and render time.

Evidence:

- a widget-tree inspector screenshot;
- damage metrics showing bounded repaint;
- runtime restart trace;
- keyboard and mouse focus behavior tests.

Lab 6: implement REPL interruption

Purpose: understand Goja ownership, context, host calls, and restart semantics.

Tasks:

88. Add a cell task ID and deadline.
89. Interrupt `while (true) {}`.
90. Cancel an awaited host operation.
91. Decide when the runtime can continue and when it must restart.
92. Replay prior cells after restart and mark ephemeral outputs stale.

Evidence:

- cell state transition trace;
- tests for synchronous loop, host I/O, and promise cancellation;
- proof that the WM loop remains responsive.

Lab 7: build the widget invalidation proof

Purpose: learn retained rendering.

Tasks:

93. Implement stable reconciliation for Text, Row, and Column.
94. Distinguish paint-only text color changes from measurement-changing font size changes.
95. Record measured nodes, laid-out nodes, and painted pixels.
96. Compare incremental output to a full rerender.

Evidence:

- a table of invalidation counts;
- pixel equality tests;
- a visual damage overlay.

Lab 8: capability and lifecycle failure injection

Purpose: treat scripts as desktop processes.

Tasks:

97. Deny a process-spawn effect and render the denial as an inspectable object.
98. Revoke a UI global-surface grant while the runtime owns a bar.
99. Disconnect a broker client during an accept and during command invocation.
100. Restart the runtime and verify registry cleanup.

Evidence:

- capability decision trace;
- no leaked surfaces, verbs, or commands;
- documented user-visible recovery behavior.

22. Architectural decisions to record now

Several decisions should be made explicitly before implementation proceeds because later code will otherwise establish them accidentally.

Decision	Recommended answer	Reason
Is go-go-wm an X11 manager with scripts, or a desktop environment with an X11 backend?	A desktop operating environment with a small, reliable X11 manager backend.	This keeps PBUI/runtime/widget contracts platform-independent while preserving a strict native core.
Does one Goja runtime own all shell and apps?	No. Use a trusted shell runtime plus supervised app, automation, and REPL runtimes.	Failure, capability, state, and hot reload boundaries must be explicit.
Can JavaScript run during paint or hit test?	No. It produces snapshots and handles posted events on its own loop.	This protects input latency and avoids cross-loop deadlocks.
Are PBUI objects values or references?	Both, represented through one reference envelope with provider and optional snapshot.	Live entities need stable identity; immutable scalars can remain self-contained.
Is accept globally singular?	No. It is scoped per seat/interaction stack with explicit policy.	Multiple runtimes and nested commands require composable interaction.
Are verbs separate from commands?	A verb is a command with a typed target.	One registry should drive menus, launcher, bindings, completion, help, and replay.
Are bars, menus, modals, and taskbars special renderers?	They are surface roles over one widget runtime.	Shared layout, damage, focus, presentations, and failure behavior reduce complexity.
Does interactive resize commit every pointer sample as a model operation?	No. Preview state is coalesced; release emits one durable operation.	This improves performance, replay, undo, and event clarity.
Must builtin/script tiles rerender at full rate during resize?	No. Geometry and content repaint have independent budgets.	Slow widgets must not block direct manipulation.
How is persistence achieved?	Versioned source, declarative registrations, serializable state, checkpoints, and replay.	Raw VM heap snapshots are fragile and mix ephemeral resources with durable state.

Decision	Recommended answer	Reason
What remains native?	X11 ownership, command/effect validation, runtime supervision, surface host, recovery shell, and protocol/resource invariants.	The system must remain recoverable when all script code is invalid or stalled.

Appendix A. Proposed protocol and API sketches

These sketches define boundaries rather than final names. They are intentionally data-oriented so the in-process and IPC implementations can share them.

A.1 Object provider

```
type ObjectProvider interface {
    Resolve(context.Context, pbui.ObjectRef) (ObjectSnapshot, error)
    Describe(context.Context, pbui.ObjectRef, InspectContext) ([]InspectorSection, error)
    Views(context.Context, pbui.ObjectRef, ViewContext) ([]ViewSpec, error)
    Commands(context.Context, pbui.ObjectRef, CommandContext) ([]CommandState, error)
    Serialize(context.Context, pbui.ObjectRef) (DurableObject, error)
}
```

Provider requests should be deadline-bound and asynchronous from JavaScript owner loops. A provider may return **gone**, **stale-revision**, **permission-denied**, or **not-serializable** as structured errors.

A.2 Command registration and invocation

```
const command = require("command");

command.define({
    id: "runtime.restart",
    title: "Restart runtime",
    targetTypes: ["runtime"],
    category: ["System", "Runtime"],
    effects: ["runtime.manage"],
    arguments: [
        {name: "replay", type: "boolean", default: true}
    ],
    enabled(ctx) {
        return ctx.target.state !== "stopped";
    }
}, async ({target, replay}) => {
    return system.runtimes.restart(target, {replay});
});
```

The host should compile declarative **enabled** conditions where possible. Arbitrary enablement functions require querying the owner runtime and should be cached with a short deadline; a slow owner must not block menu opening.

A.3 Widget and surface API

```
const ui = require("ui/v1");

const surface = ui.surface({
  id: "dev.resize-profiler",
  role: "tile",
  title: "RESIZE PROFILER",
  state: profilerState,
  render(ctx) {
    return ui.column({id: "root", gap: 8, padding: 8},
      ui.row({id: "summary", gap: 6},
        ui.metric({id: "p95", label: "P95", value: ctx.state.p95, unit: "ms"}),
        ui.metric({id: "paint", label: "PAINT", value: ctx.state.paintPixels, unit: "px"}),
        ui.metric({id: "xreq", label: "X REQS", value: ctx.state.xRequests})
      ),
      ui.linePlot({
        id: "latency",
        series: ctx.state.samples,
        x: "seq",
        y: ["inputToFlush", "paint"],
        presentation: row => pbui.ref("resize-sample", row.id)
      }),
      ui.virtualTable({
        id: "samples",
        rows: ctx.state.samples,
        key: row => row.id,
        columns: sampleColumns
      })
    );
  }
});
```

A.4 Signals and subscriptions

A small reactive state API can reduce unnecessary full render calls while keeping host control:

```
const windows = ui.resource(() => wm.windows(), {key: "wm.windows"});
const filter = ui.signal("");
const visible = ui.computed(() =>
  windows.value.filter(w => w.title.toLowerCase().includes(filter.value.toLowerCase()))
);

pbui.on("window.*", ui.invalidateResource(windows, {coalesce: "latest"}));
```

The runtime still produces a desired widget tree. Signals identify which app renders need scheduling and allow the host to coalesce refresh. Fine-grained widget damage remains a host reconciliation responsibility.

A.5 Runtime inspection API

```
const system = require("system");

const rs = await system.runtimes.list();
const slow = rs.filter(r => r.metrics.handlerP95Ms > 10);
pbui.print(...slow.map(r => pbui.ref("runtime", r.id)));

await system.runtimes.withCapability(
  runtimeRef,
  {capability: "process.spawn", scope: {commands: ["git"]}},
  async () => command.invoke("git.refresh", {target: repoRef})
);
```

Temporary grants should be explicit objects in the trace and should close when the callback settles or is cancelled.

A.6 REPL cell control

```
repl.on("cell.running", cell => {
  // A shell widget can show running tasks without holding raw VM values.
});

await repl.stop(cellRef, {graceMs: 100});
await repl.restartWorkspace({checkpoint: "last-good", replay: true});
```

The native REPL surface uses the same command protocol. Stop, Rerun, Edit, Pin, Watch, Export, Inspect Runtime, and Copy as Source are commands on `repl-cell` or `repl-result` presentations.

Appendix B. Detailed resize benchmark specification

B.1 Scripted input

A deterministic drag fixture should:

101. Position the pointer at the divider center.
102. Press the configured button.
103. Emit a known sequence of positions at a selected event rate, including reversal and snap-zone crossings.
104. Hold briefly at the end.
105. Release.
106. Wait for final paint and client settle markers.

Run event rates of 60, 120, 240, and 1000 samples per second. The manager should perform bounded commits independent of input rate and always apply the final position.

B.2 Trace points

Trace point	Data
<code>pointer.received</code>	server/event timestamp, local monotonic time, drag ID, raw coordinate, queue depth
<code>pointer.coalesced</code>	replaced sequence, newest sequence, pending count
<code>geometry.begin</code>	model revision, preview ratio, dirty root
<code>layout.end</code>	duration, nodes visited, rectangles changed
<code>x.diff.end</code>	configure/map/unmap/stack/focus effect counts
<code>x.flush</code>	duration, total request bytes if measured
<code>paint.begin/end</code>	surface, reason, damage rectangles, pixels, allocation delta
<code>upload.begin/end</code>	backend, rectangles, bytes
<code>client.configure</code>	client XID/object ref, old/new interior size
<code>drag.release</code>	final raw coordinate, committed ratio, final revision
<code>surface.settled</code>	final exact-size content revision committed

B.3 Comparison report

Each run should produce:

- latency percentile table;
- time-series plot of pointer sequence versus applied geometry sequence;
- stacked time per sample for layout, X diff, paint, upload;
- counts of coalesced samples and duplicate requests avoided;
- allocation and GC summary;
- client configure count and optional client-side redraw marker;
- final pixel comparison and geometry assertions;
- links to profiles and trace objects.

The report itself can be a persistent REPL notebook or `profile-report` presentation. That makes performance evidence directly navigable in the environment being measured.

B.4 Expected signatures

Observed signature	Likely cause	Next experiment
High WM CPU and paint pixels; outline mode fast	Manager raster/upload coupling	Suppress paint, split chrome, inspect damage.
Low WM CPU; many client configures; outline fast	Client relay layout pressure	Lower live configure rate or test another client.
Low paint; high X request count	Redundant reconciliation or mapping/stacking	Enable desired/current request diff.
Input-to-begin latency grows over drag	WM loop backlog or blocking host call	Inspect task queue and synchronous broker/JS calls.
Geometry flush is fast but visible motion lags	X server/compositor/presentation delay	Compare Xephyr/Xorg, compositor off, XSync markers.
Frequent allocations despite cached surfaces	Exact-size buffer recreation or transient slices	Add capacity reuse, pools, and allocation labels.
Final position occasionally stale	Throttle discarded last sample	Use latest-only pending state and release commit.

Appendix C. ICCCM/EWMH and desktop-integration checklist

This checklist is intentionally broader than the immediate performance work. It prevents shell UI and scripting features from growing on an incomplete interoperability foundation.

C.1 Manager ownership and startup

- ☐ Acquire `SubstructureRedirect` with a clear “another WM is running” error.
- ☐ Acquire the ICCCM manager selection for each managed screen and publish version.
- ☐ Create and maintain `_NET_SUPPORTING_WM_CHECK` and `_NET_WM_NAME`.
- ☐ Scan and manage existing eligible windows on restart/adoption.
- ☐ Distinguish override-redirect windows and never reparent them by default.
- ☐ Handle server/display shutdown without leaving socket or SHM artifacts.

C.2 Client lifecycle

- ☐ Read `WM_CLASS`, `WM_NAME`/`_NET_WM_NAME`, `WM_HINTS`, `WM_NORMAL_HINTS`, `WM_TRANSIENT_FOR`, `WM_PROTOCOLS`, and `_NET_WM_WINDOW_TYPE` before placement.
- ☐ Maintain Normal, Iconic/hidden where supported, and Withdrawn transitions consistently.
- ☐ Send `WM_DELETE_WINDOW` when supported; kill only through explicit force-close policy.
- ☐ Handle client-initiated unmap versus manager unmap without double-unmanage.
- ☐ Send synthetic `ConfigureNotify` with root-relative client geometry when requests are denied or adjusted.
- ☐ Honor size minima, maxima, increments, base size, and aspect where policy allows; expose constraint decisions in the inspector.
- ☐ Track property changes that alter title, urgency, type, transient leader, hints, or protocols.

C.3 Focus and activation

- ☐ Respect ICCCM input hints and `WM_TAKE_FOCUS` where required.
- ☐ Never set focus to `None` accidentally; use a known fallback focus window/root policy.
- ☐ Publish `_NET_ACTIVE_WINDOW` and process permitted activation requests with focus-stealing policy.
- ☐ Track urgency/demands-attention separately from focus.
- ☐ Restore focus deterministically after popup, modal, float close, fullscreen exit, workspace switch, and runtime-owned surface teardown.

C.4 Desktops and work areas

- [] Publish desktop count, names, current desktop, and per-window desktop.
- [] Publish desktop geometry and viewport according to the chosen workspace model.
- [] Publish `_NET_WORKAREA` per desktop/output policy.
- [] Support dock/window struts or explicitly reject external docks while script bars own exclusive zones.
- [] Decide sticky/all-desktop window semantics.
- [] Publish client stacking order, not only unordered client list.

C.5 Window state and operations

- [] Support `_NET_WM_STATE` protocol and fullscreen state publication/request.
- [] Support close-window client messages.
- [] Publish allowed actions based on actual window policy and hints.
- [] Decide and implement maximize semantics in a tiling environment.
- [] Decide above/below/sticky/skip-taskbar/skip-pager semantics.
- [] Support moveresize requests or document rejection and send consistent state.
- [] Keep floating, fullscreen, modal, and dock stacking layers deterministic.

C.6 Multi-output

- [] Model outputs independently from X screens.
- [] Observe XRandR topology and hotplug.
- [] Define workspace-to-output policy, focus history, and move semantics.
- [] Recompute work areas and surface anchors per output.
- [] Preserve or clamp floating geometry on output removal.
- [] Define scale/font/render policy for mixed-density outputs.
- [] Expose outputs and work areas as PBUI objects.

Appendix D. Source map

D.1 go-go-wm current code and documentation

[GG-DEV] go-go-wm developer guide, package map, ownership disciplines, test workflow, and known traps.

<https://github.com/go-go-golems/go-go-wm/blob/5b73c9f37c97538f6767ecdc3ece4fb599932377/pkg/doc/topics/developer-guide.md>

[GG-JS-API] Current JavaScript API reference for `wm`, `pbui`, `ui`, and the rich REPL.

<https://github.com/go-go-golems/go-go-wm/blob/5b73c9f37c97538f6767ecdc3ece4fb599932377/pkg/doc/topics/js-api-reference.md>

[GG-INPUT] Divider drag input path and throttling.

<https://github.com/go-go-golems/go-go-wm/blob/5b73c9f37c97538f6767ecdc3ece4fb599932377/pkg/wmx11/input.go>

[GG-MANAGE] Frame reconciliation, client configure, paint, and buffer/surface lifecycle.

<https://github.com/go-go-golems/go-go-wm/blob/5b73c9f37c97538f6767ecdc3ece4fb599932377/pkg/wmx11/manage.go>

[GG-XSHM] MIT-SHM shared pixmap upload implementation.

<https://github.com/go-go-golems/go-go-wm/blob/5b73c9f37c97538f6767ecdc3ece4fb599932377/pkg/xshm/xshm.go>

[GG-PERF] GGWM-005 profiling evidence and first performance fixes.

<https://github.com/go-go-golems/go-go-wm/blob/5b73c9f37c97538f6767ecdc3ece4fb599932377/tmp/2026/07/19/GGWM-005-PERF--rendering-performance-profiling-fast-fills-fast-x-upload-drag-throttling/design-doc/01-paint-path-performance-analysis-and-fixes.md>

[GG-XSHM-DESIGN] GGWM-006 design guide for shared pixmaps and upload-path separation.

<https://github.com/go-go-golems/go-go-wm/blob/5b73c9f37c97538f6767ecdc3ece4fb599932377/tmp/2026/07/19/GGWM-006-XSHM--zero-copy-frame-uploads-via-mit-shm-shared-pixmaps/design-doc/01-from-putimage-to-shared-pixmaps-an-intern-s-guide-to-the-x-image-upload-path-and-the-mit-shm-design.md>

[GG-FOCUS] Current focus and fullscreen state-machine ownership.

https://github.com/go-go-golems/go-go-wm/blob/5b73c9f37c97538f6767ecdc3ece4fb599932377/pkg/wmx11/focus_state.go

[GG-OBJECT] PBUI object and verb data model.

<https://github.com/go-go-golems/go-go-wm/blob/5b73c9f37c97538f6767ecdc3ece4fb599932377/pkg/pbui/object.go>

[GG-BROKER] PBUI broker, connection queues, verb ownership, accept session, and event bus.

<https://github.com/go-go-golems/go-go-wm/blob/5b73c9f37c97538f6767ecdc3ece4fb599932377/pkg/pbui/broker/broker.go>

[GG-JSBRIDGE] JavaScript bridge, owner-loop boundaries, queues, and event fan-out.

<https://github.com/go-go-golems/go-go-wm/tree/5b73c9f37c97538f6767ecdc3ece4fb599932377/pkg/jsmod>

[GG-UI] Script UI app and normalized snapshot implementation.

<https://github.com/go-go-golems/go-go-wm/blob/5b73c9f37c97538f6767ecdc3ece4fb599932377/pkg/jsmod/uimod/app.go>

[GG-UI-DOC] Current UI module contract and concurrency shape.

<https://github.com/go-go-golems/go-go-wm/blob/5b73c9f37c97538f6767ecdc3ece4fb599932377/pkg/doc/topics/ui-module.md>

[GG-REPL-DOC] GGWM-009 rich REPL design.

<https://github.com/go-go-golems/go-go-wm/blob/5b73c9f37c97538f6767ecdc3ece4fb599932377/tmp/2026/07/19/GGWM-009-RICH-REPL--a-rich-widget-pbui-repl-wolfram-style-values-as-live-presentations/design-doc/01-the-rich-repl-an-intern-s-guide-to-wolfram-style-values-as-pbui-presentations.md>

[GG-SCRIPTING] WM in-process scripting backend and owner-loop posting.

<https://github.com/go-go-golems/go-go-wm/blob/5b73c9f37c97538f6767ecdc3ece4fb599932377/pkg/wmx11/scripting.go>

D.2 Attached design prototypes

[PROTO-SHELL] PBUI shell prototype: typed live presentations, accept across tiles/workspaces, object menus, split tree, and shared app state. `pbui-shell(3).jsx`, attached to the review.

[PROTO-BASKETBALL] Basketball PBUI workbench: linked typed player/team/game objects across table, shot chart, radar, trends, scatter, standings, watchlist, inspector, and trace. `pbui-basketball.jsx`, attached to the review.

D.3 Other window managers and UI systems

[I3-HACK] i3 hacking guide, layout tree and render-to-X responsibilities.

<https://github.com/i3/i3/blob/f7d5b8983f6caeab94585e5359d7de59e7041e9b/docs/hacking-howto>

[I3-X] i3 X state cache and change-pushing implementation.

<https://github.com/i3/i3/blob/f7d5b8983f6caeab94585e5359d7de59e7041e9b/src/x.c>

[I3-DEC0] i3 decoration render-parameter caching in `src/x.c`.

<https://github.com/i3/i3/blob/f7d5b8983f6caeab94585e5359d7de59e7041e9b/src/x.c>

[SWAY-TXN] Sway dirty-node transaction queue and pending/current state application.

<https://github.com/swaywm/sway/blob/6959a78a8f0d52f79ad7465135b3295307a5146a/sway/desktop/transaction.c>

[AWESOME-WIDGET] AwesomeWM declarative widget construction, stable IDs, caches, and separate layout/redraw signals.

<https://github.com/awesomeWM/awesome/blob/39419132eb7b0ceb886d8e55b5d80dcf86e86647/lib/wibox/widget/base.lua>

[QTILE-CMD] Qtile command graph, command objects, interfaces, IPC, and lazy/live clients.

<https://github.com/qtile/qtile/blob/8bde8a1bb770704ecb3e27cf3b6a57c4ad3e08dd/docs/manual/commands/advanced.rst>

[XMONAD] XMonad's minimal stable core plus extension-library model.

<https://github.com/xmonad/xmonad/blob/master/README.md>

D.4 Platform and historical interaction sources

[X11] Xlib reference for configuring mapped windows, `SubstructureRedirect`, `ConfigureRequest`, and `Expose` behavior.

<https://www.x.org/releases/X11R7.6/doc/libX11/specs/libX11/libX11.html>

[ICCCM] ICCCM client-to-window-manager communication and property conventions.

<https://tronche.com/gui/x/icccm/sec-4.html>

[EWMH] Extended Window Manager Hints specification.

<https://specifications.freedesktop.org/wm/wm-spec-latest.html>

[CLIM] McCLIM manual sections on presentation types, typed input contexts, and command tables.

<https://mcclim.common-lisp.dev/static/manual/mcclim.html>

[HYPERCARD] HyperTalk message-passing order through button/field, card, background, stack, Home stack, and HyperCard.

<https://www.hypercard.center/HyperTalkReference/hypertalkbasics/The-message-passing-order>

[SMALLTALK] Pharo description of a live environment with immediate feedback and an integrated debugger, together with Squeak/Pharo educational material.

<https://www.pharo-project.org/>

<https://books.pharo.org/pharo-by-example9/>

Appendix E. File-by-file review prompts for future contributors

Before changing a subsystem, use these questions to keep the current architectural disciplines intact.

pkg/wmcore

- Is the behavior deterministic without X11, time, or goroutines?
- Is mutation represented by one serializable operation?
- Does the operation report enough dirty information for platform reconciliation?
- Can a generated sequence test the invariant?
- Are node IDs stable through moves and unique through clones?

pkg/wmx11

- Which loop owns the state being changed?
- Is this logical decision testable before an X request is sent?
- Does desired/current state prevent duplicate requests?
- Does this path call paint, map, stack, or EWMH updates more broadly than necessary?
- What happens when the client disappears between decision and request?
- Is teardown idempotent after partial construction?
- Does fullscreen, floating, modal, popup, or workspace state override this path?

pkg/draw and pkg/xshm

- What exact region changed?
- Can the backing store remain allocated and retained?
- Is a full image conversion necessary?
- Does a size change alter pixels or only placement/clipping?
- Is the upload backend independent from widget invalidation?
- Are server-side references detached before resource destruction?

pkg/pbui and broker

- Is identity stable and namespaced?
- Is the message reliable, latest-state, canonical-stream, or telemetry?
- What happens when the owner disconnects or restarts?
- Is authorization checked at registration and invocation?
- Can the interaction nest or coexist with another seat/runtime?
- Does a stale object revision fail explicitly?

pkg/jsmod and xgoja providers

- Does any foreign loop call Goja directly?
- Can any host call block the owner loop?
- Is module/provider state local to one runtime instance?
- Are exported values plain, bounded, and validated?
- Can the task be cancelled and forcibly interrupted?
- What resources are registered, and how are they removed on runtime close?
- Which capability authorizes the effect?

pkg/repl

- Is every cell transition explicit and inspectable?
- Can Stop interrupt synchronous JS and host I/O?
- Can the workspace restart and replay deterministically?
- Is text editing Unicode-safe and non-blocking?
- Is result identity independent from string formatting?
- Are ephemeral values marked when their provider/runtime dies?

- Can the result be inspected, accepted, watched, serialized, or converted to source?

Closing assessment

The project does not need to choose between being a fast window manager and being an experimental presentation-based environment. It needs a hard boundary between the two. The native manager should be small, state-machine-driven, transaction-based, protocol-compliant, and resistant to script failure. The environment above it should be live, typed, inspectable, and highly programmable.

The current code already contains the critical seeds: operations as data, owner loops, normalized render snapshots, a broker independent of X11, live typed values, type-directed commands, and a rich REPL. The next step is not to add more callbacks. It is to turn those seeds into explicit services: desired/current geometry reconciliation, retained surfaces, stable object providers, one command graph, scoped interaction contexts, supervised runtimes, capabilities, persistence, and an inspector that can traverse all of them.

Done in that order, resize performance improves because geometry no longer implies full paint. UI scriptability improves because widgets are retained data with stable identity rather than full-surface rerenders. Reliability improves because every runtime is interruptible and independently recoverable. The REPL becomes the place where the desktop understands itself. That combination is the novel system go-go-wm is positioned to build.